Standard Semirings

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Non Negative Integer Semiring 1

2 Tropical Semirings 3

3 Univariate Tropical Polynomials 9

4 Multivariate Tropical Polynomials 25

5 Matrices over tropical semirings 45

6 Tropical Varieties 49

7 Indices and Tables 67

Python Module Index 69

Index 71

CHAPTER

ONE

NON NEGATIVE INTEGER SEMIRING

class sage.rings.semirings.non_negative_integer_semiring.**NonNegativeIntegerSemiring**

Bases: NonNegativeIntegers

A class for the semiring of the nonnegative integers.

This parent inherits from the infinite enumerated set of non negative integers and endows it with its natural semiring structure.

EXAMPLES:

```
sage: NonNegativeIntegerSemiring()
Non negative integer semiring
```

```
>>> from sage.all import *
>>> NonNegativeIntegerSemiring()
Non negative integer semiring
```

For convenience, NN is a shortcut for NonNegativeIntegerSemiring():

```
sage: NN == NonNegativeIntegerSemiring()
True

sage: NN.category()
Category of facade infinite enumerated commutative semirings
```

```
>>> from sage.all import *
>>> NN == NonNegativeIntegerSemiring()
True

>>> NN.category()
Category of facade infinite enumerated commutative semirings
```

Here is a piece of the Cayley graph for the multiplicative structure:

```
sage: G = NN.cayley_graph(elements=range(9), generators=[0,1,2,3,5,7])          #␣
↪needs sage.graphs
sage: G                                                                          #␣
↪needs sage.graphs
Looped multi-digraph on 9 vertices
sage: G.plot()                                                                   #␣
↪needs sage.graphs sage.plot
Graphics object consisting of 48 graphics primitives
```

```
>>> from sage.all import *
>>> G = NN.cayley_graph(elements=range(Integer(9)), generators=[Integer(0),
↪Integer(1),Integer(2),Integer(3),Integer(5),Integer(7)])          # needs sage.
↪graphs
>>> G                                                                          #␣
↪needs sage.graphs
Looped multi-digraph on 9 vertices
>>> G.plot()                                                                   #␣
↪needs sage.graphs sage.plot
Graphics object consisting of 48 graphics primitives
```

This is the Hasse diagram of the divisibility order on `NN`.

sage: Poset(NN.cayley_graph(elements=[1..12], generators=[2,3,5,7,11])).show() # needs sage.combinat sage.graphs sage.plot

Note: as for `NonNegativeIntegers`, `NN` is currently just a "facade" parent; namely its elements are plain Sage `Integers` with `Integer Ring` as parent:

```
sage: x = NN(15); type(x)
<class 'sage.rings.integer.Integer'>
sage: x.parent()
Integer Ring
sage: x+3
18
```

```
>>> from sage.all import *
>>> x = NN(Integer(15)); type(x)
<class 'sage.rings.integer.Integer'>
>>> x.parent()
Integer Ring
>>> x+Integer(3)
18
```

additive_semigroup_generators()

Return the additive semigroup generators of `self`.

EXAMPLES:

```
sage: NN.additive_semigroup_generators()
Family (0, 1)
```

```
>>> from sage.all import *
>>> NN.additive_semigroup_generators()
Family (0, 1)
```

CHAPTER

TWO

TROPICAL SEMIRINGS

AUTHORS:

- Travis Scrimshaw (2013-04-28) - Initial version

class sage.rings.semirings.tropical_semiring.**TropicalSemiring**(*base*, *use_min=True*)

Bases: Parent, UniqueRepresentation

The tropical semiring.

Given an ordered additive semigroup R, we define the tropical semiring $T = R \cup \{+\infty\}$ by defining tropical addition and multiplication as follows:

$$a \oplus b = \min(a, b), \qquad a \odot b = a + b.$$

In particular, note that there are no (tropical) additive inverses (except for ∞), and every element in R has a (tropical) multiplicative inverse.

There is an alternative definition where we define $T = R \cup \{-\infty\}$ and alter tropical addition to be defined by

$$a \oplus b = \max(a, b).$$

To use the max definition, set the argument `use_min = False`.

Warning

zero() and *one()* refer to the tropical additive and multiplicative identities respectively. These are **not** the same as calling `T(0)` and `T(1)` respectively as these are **not** the tropical additive and multiplicative identities respectively.

Specifically do not use `sum(...)` as this converts 0 to 0 as a tropical element, which is not the same as *zero()*. Instead use the `sum` method of the tropical semiring:

```
sage: T = TropicalSemiring(QQ)

sage: sum([T(1), T(2)]) # This is wrong
0
sage: T.sum([T(1), T(2)]) # This is correct
1
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)

>>> sum([T(Integer(1)), T(Integer(2))]) # This is wrong
0
>>> T.sum([T(Integer(1)), T(Integer(2))]) # This is correct
1
```

Be careful about using code that has not been checked for tropical safety.

INPUT:

- `base` – the base ordered additive semigroup R
- `use_min` – boolean (default: `True`); if `True`, then the semiring uses $a \oplus b = \min(a, b)$. Otherwise uses $a \oplus b = \max(a, b)$.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: elt = T(2); elt
2
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> elt = T(Integer(2)); elt
2
```

Recall that tropical addition is the minimum of two elements:

```
sage: T(3) + T(5)
3
```

```
>>> from sage.all import *
>>> T(Integer(3)) + T(Integer(5))
3
```

Tropical multiplication is the addition of two elements:

```
sage: T(2) * T(3)
5
sage: T(0) * T(-2)
-2
```

```
>>> from sage.all import *
>>> T(Integer(2)) * T(Integer(3))
5
>>> T(Integer(0)) * T(-Integer(2))
-2
```

We can also do tropical division and arbitrary tropical exponentiation:

```
sage: T(2) / T(1)
1
sage: T(2)^(-3/7)
-6/7
```

```
>>> from sage.all import *
>>> T(Integer(2)) / T(Integer(1))
1
>>> T(Integer(2))**(-Integer(3)/Integer(7))
-6/7
```

Note that "zero" and "one" are the additive and multiplicative identities of the tropical semiring. In general, they are **not** the elements 0 and 1 of R, respectively, even if such elements exist (e.g., for $R = \mathbf{Z}$), but instead the (tropical) additive and multiplicative identities $+\infty$ and 0 respectively:

```
sage: T.zero() + T(3) == T(3)
True
sage: T.one() * T(3) == T(3)
True
sage: T.zero() == T(0)
False
sage: T.one() == T(1)
False
```

```
>>> from sage.all import *
>>> T.zero() + T(Integer(3)) == T(Integer(3))
True
>>> T.one() * T(Integer(3)) == T(Integer(3))
True
>>> T.zero() == T(Integer(0))
False
>>> T.one() == T(Integer(1))
False
```

Element

alias of *TropicalSemiringElement*

additive_identity()

alias of *zero()*.

gens()

Return the generators of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: T.gens()
(1, +infinity)
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> T.gens()
(1, +infinity)
```

infinity()

alias of *zero()*.

multiplicative_identity()

alias of *one()*.

one()

Return the (tropical) multiplicative identity element 0.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: T.one()
0
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> T.one()
0
```

zero()

Return the (tropical) additive identity element $+\infty$.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: T.zero()
+infinity
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> T.zero()
+infinity
```

class sage.rings.semirings.tropical_semiring.**TropicalSemiringElement**

Bases: Element

An element in the tropical semiring over an ordered additive semigroup R. Either in R or ∞. The operators $+, \cdot$ are defined as the tropical operators $\oplus, \odot$ respectively.

lift()

Return the value of self lifted to the base.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: elt = T(2)
sage: elt.lift()
2
sage: elt.lift().parent() is QQ
True
sage: T.additive_identity().lift().parent()
The Infinity Ring
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> elt = T(Integer(2))
>>> elt.lift()
2
>>> elt.lift().parent() is QQ
True
>>> T.additive_identity().lift().parent()
The Infinity Ring
```

multiplicative_order()

Return the multiplicative order of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: T.multiplicative_identity().multiplicative_order()
1
sage: T.additive_identity().multiplicative_order()
+Infinity
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> T.multiplicative_identity().multiplicative_order()
1
>>> T.additive_identity().multiplicative_order()
+Infinity
```

class sage.rings.semirings.tropical_semiring.**TropicalToTropical**

Bases: Map

Map from the tropical semiring to itself (possibly with different bases). Used in coercion.

CHAPTER

THREE

UNIVARIATE TROPICAL POLYNOMIALS

AUTHORS:

- Verrel Rievaldo Wijaya (2024-06): initial version

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x> = PolynomialRing(T)
sage: x.parent()
Univariate Tropical Polynomial Semiring in x over Rational Field
sage: (x + R(3)*x) * (x^2 + x)
3*x^3 + 3*x^2
sage: (x^2 + R(1)*x + R(-1))^2
0*x^4 + 1*x^3 + 2*x^2 + 0*x + (-2)
sage: (x^2 + x + R(0))^4
0*x^8 + 0*x^7 + 0*x^6 + 0*x^5 + 0*x^4 + 0*x^3 + 0*x^2 + 0*x + 0
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> x.parent()
Univariate Tropical Polynomial Semiring in x over Rational Field
>>> (x + R(Integer(3))*x) * (x**Integer(2) + x)
3*x^3 + 3*x^2
>>> (x**Integer(2) + R(Integer(1))*x + R(-Integer(1)))**Integer(2)
0*x^4 + 1*x^3 + 2*x^2 + 0*x + (-2)
>>> (x**Integer(2) + x + R(Integer(0)))**Integer(4)
0*x^8 + 0*x^7 + 0*x^6 + 0*x^5 + 0*x^4 + 0*x^3 + 0*x^2 + 0*x + 0
```

REFERENCES:

- [Bru2014]
- [Fil2017]

class sage.rings.semirings.tropical_polynomial.**TropicalPolynomial**(*parent*, *x=None*, *check=True*, *is_gen=False*, *construct=False*)

Bases: Polynomial_generic_sparse

A univariate tropical polynomial.

A tropical polynomial is a polynomial whose coefficients come from a tropical semiring. Tropical polynomial induces a function which is piecewise linear, where each segment of the function has a non-negative integer slope.

Tropical roots (zeros) of polynomial $P(x)$ is defined as all points x_0 for which the graph of $P(x)$ change its slope. The difference in the slopes of the two pieces adjacent to this root gives the order of the root.

The tropical polynomials are implemented with a sparse format by using a `dict` whose keys are the exponent and values the corresponding coefficients. Each coefficient is a tropical number.

EXAMPLES:

We construct a tropical polynomial semiring by defining a base tropical semiring and then inputting it to `PolynomialRing()`:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x> = PolynomialRing(T); R
Univariate Tropical Polynomial Semiring in x over Rational Field
sage: R.0
0*x
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1); R
Univariate Tropical Polynomial Semiring in x over Rational Field
>>> R.gen(0)
0*x
```

One way to construct an element is to provide a list or tuple of coefficients. Another way to define an element is to write a polynomial equation with each coefficient converted to the semiring:

```
sage: p1 = R([1,4,None,0]); p1
0*x^3 + 4*x + 1
sage: p2 = R(1)*x^2 + R(2)*x + R(3); p2
1*x^2 + 2*x + 3
```

```
>>> from sage.all import *
>>> p1 = R([Integer(1),Integer(4),None,Integer(0)]); p1
0*x^3 + 4*x + 1
>>> p2 = R(Integer(1))*x**Integer(2) + R(Integer(2))*x + R(Integer(3)); p2
1*x^2 + 2*x + 3
```

We can do some basic arithmetic operations for these tropical polynomials. Remember that numbers given have to be in the tropical semiring. If not, then it will raise an error:

```
sage: p1 + p2
0*x^3 + 1*x^2 + 4*x + 3
sage: p1 * p2
1*x^5 + 2*x^4 + 5*x^3 + 6*x^2 + 7*x + 4
sage: 2 * p1
Traceback (most recent call last):
...
ArithmeticError: cannot negate any non-infinite element
sage: T(2) * p1
2*x^3 + 6*x + 3
sage: p1(3)
Traceback (most recent call last):
...
TypeError: no common canonical parent for objects with parents:
```

(continues on next page)

(continued from previous page)

```
'Tropical semiring over Rational Field' and 'Integer Ring'
sage: p1(T(3))
9
```

```
>>> from sage.all import *
>>> p1 + p2
0*x^3 + 1*x^2 + 4*x + 3
>>> p1 * p2
1*x^5 + 2*x^4 + 5*x^3 + 6*x^2 + 7*x + 4
>>> Integer(2) * p1
Traceback (most recent call last):
...
ArithmeticError: cannot negate any non-infinite element
>>> T(Integer(2)) * p1
2*x^3 + 6*x + 3
>>> p1(Integer(3))
Traceback (most recent call last):
...
TypeError: no common canonical parent for objects with parents:
'Tropical semiring over Rational Field' and 'Integer Ring'
>>> p1(T(Integer(3)))
9
```

We can find all tropical roots of a tropical polynomial, counted with their multiplicities:

```
sage: p1.roots()
[-3, 2, 2]
sage: p2.roots()
[1, 1]
```

```
>>> from sage.all import *
>>> p1.roots()
[-3, 2, 2]
>>> p2.roots()
[1, 1]
```

Even though every tropical polynomials have tropical roots, this does not necessarily means it can be factored into its linear factors:

```
sage: p1.factor()
(0) * (0*x^3 + 4*x + 1)
sage: p2.factor()
(1) * (0*x + 1)^2
```

```
>>> from sage.all import *
>>> p1.factor()
(0) * (0*x^3 + 4*x + 1)
>>> p2.factor()
(1) * (0*x + 1)^2
```

Every tropical polynomial $p(x)$ have a corresponding unique tropical polynomial $\bar{p}(x)$ with the same roots that can be factored. We call $\bar{p}(x)$ the tropical polynomial split form of $p(x)$:

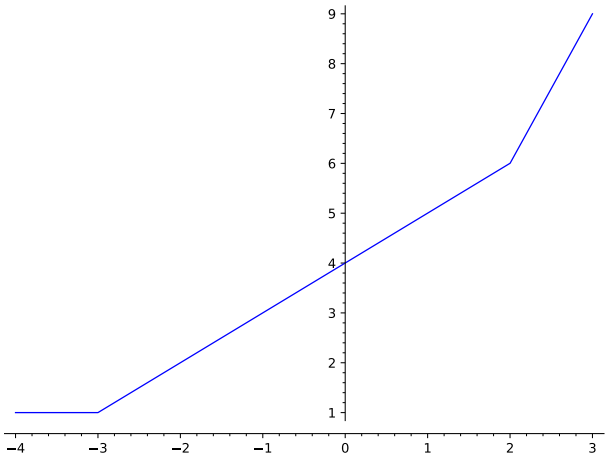
```
sage: p1.split_form()
0*x^3 + 2*x^2 + 4*x + 1
sage: p2.split_form()
1*x^2 + 2*x + 3
```

```
>>> from sage.all import *
>>> p1.split_form()
0*x^3 + 2*x^2 + 4*x + 1
>>> p2.split_form()
1*x^2 + 2*x + 3
```

Every tropical polynomial induce a piecewise linear function that can be invoked in the following way:

```
sage: p1.piecewise_function()
piecewise(x|-->1 on (-oo, -3], x|-->x + 4 on (-3, 2), x|-->3*x on [2, +oo); x)
sage: p1.plot()
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> p1.piecewise_function()
piecewise(x|-->1 on (-oo, -3], x|-->x + 4 on (-3, 2), x|-->3*x on [2, +oo); x)
>>> p1.plot()
Graphics object consisting of 1 graphics primitive
```

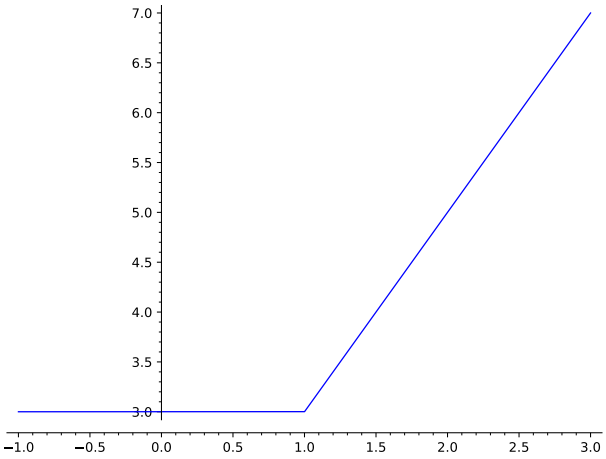



```
sage: p2.piecewise_function()
piecewise(x|-->3 on (-oo, 1], x|-->2*x + 1 on (1, +oo); x)
sage: p2.plot()
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> p2.piecewise_function()
piecewise(x|-->3 on (-oo, 1], x|-->2*x + 1 on (1, +oo); x)
>>> p2.plot()
Graphics object consisting of 1 graphics primitive
```

factor()

Return the factorization of `self` into its tropical linear factors.



Note that the factor $x - x_0$ in classical algebra gets transformed to the factor $x + x_0$, since the root of the tropical polynomial $x + x_0$ is x_0 and not $-x_0$. However, not every tropical polynomial can be factored.

OUTPUT: :class:'Factorization'

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=True)
sage: R.<x> = PolynomialRing(T)
sage: p1 = R([6,3,1,0]); p1
0*x^3 + 1*x^2 + 3*x + 6
sage: factor(p1)
(0) * (0*x + 1) * (0*x + 2) * (0*x + 3)
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=True)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> p1 = R([Integer(6),Integer(3),Integer(1),Integer(0)]); p1
0*x^3 + 1*x^2 + 3*x + 6
>>> factor(p1)
(0) * (0*x + 1) * (0*x + 2) * (0*x + 3)
```

Such factorization is not always possible:

```
sage: p2 = R([4,4,2]); p2
2*x^2 + 4*x + 4
sage: p2.factor()
(2) * (0*x^2 + 2*x + 2)
```

```
>>> from sage.all import *
>>> p2 = R([Integer(4),Integer(4),Integer(2)]); p2
2*x^2 + 4*x + 4
>>> p2.factor()
(2) * (0*x^2 + 2*x + 2)
```

piecewise_function()

Return the tropical polynomial function of `self`.

The function is a piecewise linear function with the domains are divided by the roots. First we convert each

term of polynomial to its corresponding linear function. Next, we must determine which term achieves the minimum (maximum) at each interval.

OUTPUT: a piecewise function

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x> = PolynomialRing(T)
sage: p1 = R([4,2,1,3]); p1
3*x^3 + 1*x^2 + 2*x + 4
sage: p1.piecewise_function()
piecewise(x|-->4 on (-oo, 1/3], x|-->3*x + 3 on (1/3, +oo); x)
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> p1 = R([Integer(4),Integer(2),Integer(1),Integer(3)]); p1
3*x^3 + 1*x^2 + 2*x + 4
>>> p1.piecewise_function()
piecewise(x|-->4 on (-oo, 1/3], x|-->3*x + 3 on (1/3, +oo); x)
```

A constant tropical polynomial will result in a constant function:

```
sage: p2 = R(3)
sage: p2.piecewise_function()
3
```

```
>>> from sage.all import *
>>> p2 = R(Integer(3))
>>> p2.piecewise_function()
3
```

A monomial will resulted in a linear function:

```
sage: p3 = R(1)*x^3
sage: p3.piecewise_function()
3*x + 1
```

```
>>> from sage.all import *
>>> p3 = R(Integer(1))*x**Integer(3)
>>> p3.piecewise_function()
3*x + 1
```

plot(*xmin=None*, *xmax=None*)

Return the plot of `self`, which is the tropical polynomial function we get from `self.piecewise_function()`.

INPUT:

- `xmin` – (optional) real number
- `xmax` – (optional) real number

OUTPUT:

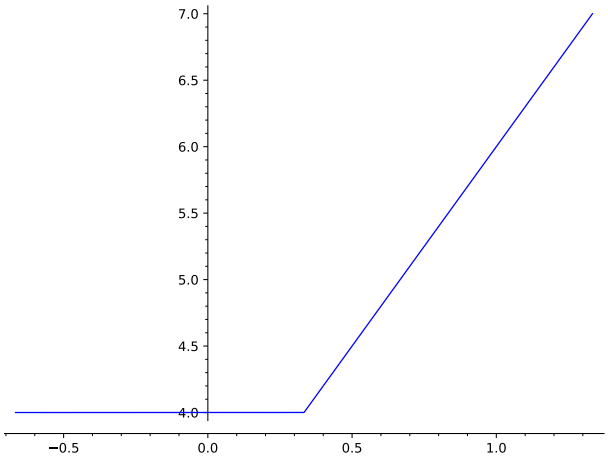
If `xmin` and `xmax` is given, then it return a plot of piecewise linear function of `self` with the axes start from `xmin` to `xmax`. Otherwise, the domain will start from the the minimum root of `self` minus 1 to maximum root of `self` plus 1. If the function of `self` is constant or linear, then the default domain will be $[-1, 1]$.

EXAMPLES:

If the tropical semiring use a max-plus algebra, then the graph will be of piecewise linear convex function:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x> = PolynomialRing(T)
sage: p1 = R([4,2,1,3]); p1
3*x^3 + 1*x^2 + 2*x + 4
sage: p1.roots()
[1/3, 1/3, 1/3]
sage: p1.plot()
Graphics object consisting of 1 graphics primitive
```

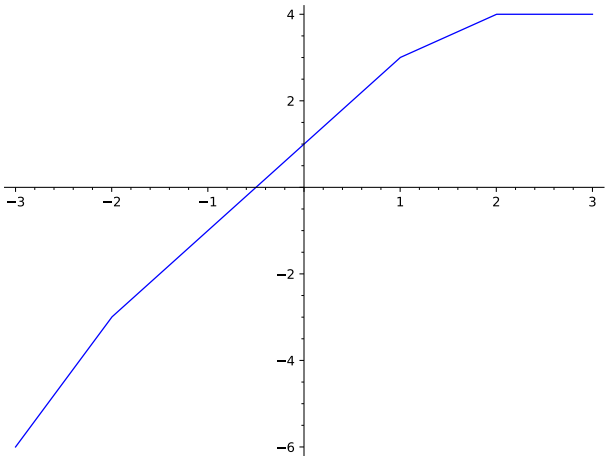
```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> p1 = R([Integer(4),Integer(2),Integer(1),Integer(3)]); p1
3*x^3 + 1*x^2 + 2*x + 4
>>> p1.roots()
[1/3, 1/3, 1/3]
>>> p1.plot()
Graphics object consisting of 1 graphics primitive
```



A different result will be obtained if the tropical semiring employs a min-plus algebra. Rather, a graph of the piecewise linear concave function will be obtained:

```
sage: T = TropicalSemiring(QQ, use_min=True)
sage: R.<x> = PolynomialRing(T)
sage: p1 = R([4,2,1,3])
sage: p1.roots()
[-2, 1, 2]
sage: p1.plot()
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=True)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> p1 = R([Integer(4),Integer(2),Integer(1),Integer(3)])
>>> p1.roots()
[-2, 1, 2]
>>> p1.plot()
Graphics object consisting of 1 graphics primitive
```



roots()

Return the list of all tropical roots of `self`, counted with multiplicity.

OUTPUT: a list of tropical numbers

ALGORITHM:

For each pair of monomials in the polynomial, we find the point where their values are equal. This is the same as solving the equation $c_1 + a_1 \cdot x = c_2 + a_2 \cdot x$ for x, where (c_1, a_1) and (c_2, a_2) are the coefficients and exponents of monomials.

The solution to this equation is $x = (c_2 - c_1)/(a_1 - a_2)$. We substitute this x to each monomials in polynomial and check if the maximum (minimum) is achieved by the previous two monomials. If it is, then x is the root of tropical polynomial. In this case, the order of the root at x is the maximum of $|i - j|$ for all possible pairs i, j which realise this maximum (minimum).

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=True)
sage: R.<x> = PolynomialRing(T)
sage: p1 = R([5,4,1,0,2,4,3]); p1
3*x^6 + 4*x^5 + 2*x^4 + 0*x^3 + 1*x^2 + 4*x + 5
sage: p1.roots()
[-1, -1, -1, 1, 2, 2]
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=True)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> p1 = R([Integer(5),Integer(4),Integer(1),Integer(0),Integer(2),Integer(4),
↪Integer(3)]); p1
```

(continues on next page)

(continued from previous page)

```
3*x^6 + 4*x^5 + 2*x^4 + 0*x^3 + 1*x^2 + 4*x + 5
>>> p1.roots()
[-1, -1, -1, 1, 2, 2]
```

There will be no tropical root for constant polynomial. Additionaly, for a monomial, the tropical root is assumed to be the additive identity of its base tropical semiring:

```
sage: p2 = R(2)
sage: p2.roots()
[]
sage: p3 = x^3
sage: p3.roots()
[+infinity, +infinity, +infinity]
```

```
>>> from sage.all import *
>>> p2 = R(Integer(2))
>>> p2.roots()
[]
>>> p3 = x**Integer(3)
>>> p3.roots()
[+infinity, +infinity, +infinity]
```

split_form()

Return the tropical polynomial which has the same roots as `self` but which can be reduced to its linear factors.

If a tropical polynomial has roots at $x_1, x_2, \ldots, x_n$, then its split form is the tropical product of linear terms of the form $(x + x_i)$ for all $i = 1, 2, \ldots, n$.

OUTPUT: *TropicalPolynomial*

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=True)
sage: R.<x> = PolynomialRing(T)
sage: p1 = R([5,4,1,0,2,4,3]); p1
3*x^6 + 4*x^5 + 2*x^4 + 0*x^3 + 1*x^2 + 4*x + 5
sage: p1.split_form()
3*x^6 + 2*x^5 + 1*x^4 + 0*x^3 + 1*x^2 + 3*x + 5
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=True)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> p1 = R([Integer(5),Integer(4),Integer(1),Integer(0),Integer(2),Integer(4),
↪Integer(3)]); p1
3*x^6 + 4*x^5 + 2*x^4 + 0*x^3 + 1*x^2 + 4*x + 5
>>> p1.split_form()
3*x^6 + 2*x^5 + 1*x^4 + 0*x^3 + 1*x^2 + 3*x + 5
```

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x> = PolynomialRing(T)
sage: p1 = R([5,4,1,0,2,4,3])
```

(continues on next page)

(continued from previous page)

```
sage: p1.split_form()
3*x^6 + 4*x^5 + 21/5*x^4 + 22/5*x^3 + 23/5*x^2 + 24/5*x + 5
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> p1 = R([Integer(5),Integer(4),Integer(1),Integer(0),Integer(2),Integer(4),
↪Integer(3)])
>>> p1.split_form()
3*x^6 + 4*x^5 + 21/5*x^4 + 22/5*x^3 + 23/5*x^2 + 24/5*x + 5
```

class sage.rings.semirings.tropical_polynomial.**TropicalPolynomialSemiring**(*base_semiring*, *names*)

Bases: UniqueRepresentation, Parent

The semiring of univariate tropical polynomials.

This is the commutative semiring consisting of finite linear combinations of tropical monomials under (tropical) addition and multiplication with the identity element being $+\infty$ (or $-\infty$ depending on whether the base tropical semiring is using min-plus or max-plus algebra).

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x> = PolynomialRing(T)
sage: f = T(1)*x
sage: g = T(2)*x
sage: f + g
1*x
sage: f * g
3*x^2
sage: f + R.zero() == f
True
sage: f * R.zero() == R.zero()
True
sage: f * R.one() == f
True
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x',)); (x,) = R._first_ngens(1)
>>> f = T(Integer(1))*x
>>> g = T(Integer(2))*x
>>> f + g
1*x
>>> f * g
3*x^2
>>> f + R.zero() == f
True
>>> f * R.zero() == R.zero()
True
>>> f * R.one() == f
True
```

Element

alias of *TropicalPolynomial*

gen(*n=0*)

Return the indeterminate generator of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<abc> = PolynomialRing(T)
sage: R.gen()
0*abc
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('abc',)); (abc,) = R._first_ngens(1)
>>> R.gen()
0*abc
```

gens()

Return the generators for `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<abc> = PolynomialRing(T)
sage: R.gens()
(0*abc,)
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('abc',)); (abc,) = R._first_ngens(1)
>>> R.gens()
(0*abc,)
```

interpolation(*points*)

Return a tropical polynomial with its function is a linear interpolation of point in `points` if possible.

If there is only one point, then it will give a constant polynomial. Because we are using linear interpolation, each point is actually a root of the resulted tropical polynomial.

INPUT:

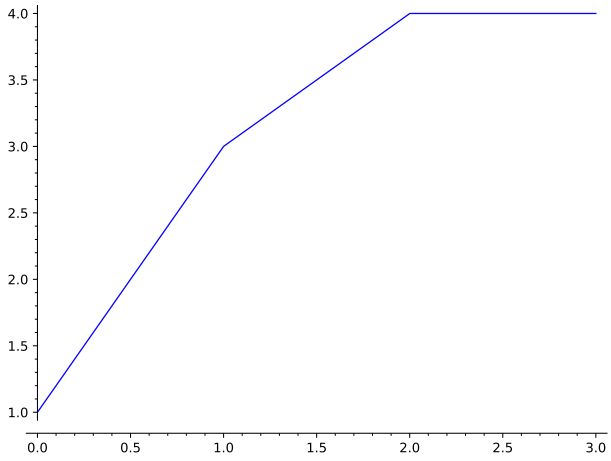
- `points` – a list of tuples `(x, y)`

OUTPUT: *TropicalPolynomial*

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=True)
sage: R = PolynomialRing(T, 'x')
sage: points = [(-2,-3),(1,3),(2,4)]
sage: p1 = R.interpolation(points); p1
1*x^2 + 2*x + 4
sage: p1.plot()
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=True)
>>> R = PolynomialRing(T, 'x')
>>> points = [(-Integer(2),-Integer(3)),(Integer(1),Integer(3)),(Integer(2),
↪Integer(4))]
>>> p1 = R.interpolation(points); p1
1*x^2 + 2*x + 4
>>> p1.plot()
Graphics object consisting of 1 graphics primitive
```




```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R = PolynomialRing(T,'x')
sage: points = [(0,0),(1,1),(2,4)]
sage: p1 = R.interpolation(points); p1
(-2)*x^3 + (-1)*x^2 + 0*x + 0
sage: p1.plot()
Graphics object consisting of 1 graphics primitive
```

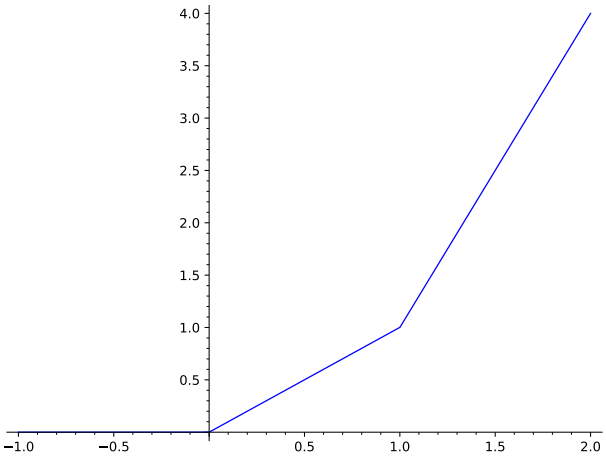
```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T,'x')
>>> points = [(Integer(0),Integer(0)),(Integer(1),Integer(1)),(Integer(2),
↪Integer(4))]
>>> p1 = R.interpolation(points); p1
(-2)*x^3 + (-1)*x^2 + 0*x + 0
>>> p1.plot()
Graphics object consisting of 1 graphics primitive
```

is_sparse()

Return `True` to indicate that the objects are sparse polynomials.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R = PolynomialRing(T, 'x')
sage: R.is_sparse()
True
```




```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, 'x')
>>> R.is_sparse()
True
```

ngens()

Return the number of generators of `self`, which is 1 since it is a univariate polynomial ring.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R = PolynomialRing(T, 'x')
sage: R.ngens()
1
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, 'x')
>>> R.ngens()
1
```

one()

Return the multiplicative identity of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R = PolynomialRing(T, 'x')
sage: R.one()
0
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, 'x')
>>> R.one()
0
```

random_element(*degree=(-1, 2)*, *monic=False*, **args*, ***kwds*)

Return a random tropical polynomial of given degrees (bounds).

OUTPUT: *TropicalPolynomial*

See also

sage.rings.polynomial.polynomial_ring.PolynomialRing_generic.random_element()

EXAMPLES:

Tropical polynomial over an integer with each coefficient bounded between `x` and `y`:

```
sage: T = TropicalSemiring(ZZ)
sage: R = PolynomialRing(T, 'x')
sage: f = R.random_element(8, x=3, y=10)
sage: f.degree()
8
sage: f.parent() is R
True
sage: all(a >= T(3) for a in f.coefficients())
True
sage: all(a < T(10) for a in f.coefficients())
True
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(ZZ)
>>> R = PolynomialRing(T, 'x')
>>> f = R.random_element(Integer(8), x=Integer(3), y=Integer(10))
>>> f.degree()
8
>>> f.parent() is R
True
>>> all(a >= T(Integer(3)) for a in f.coefficients())
True
>>> all(a < T(Integer(10)) for a in f.coefficients())
True
```

If a tuple of two integers is provided for the `degree` argument, a polynomial is selected with degrees within that range:

```
sage: all(R.random_element(degree=(1, 5)).degree() in range(1, 6) for _ in␣
↪range(10^3))
True
```

```
>>> from sage.all import *
>>> all(R.random_element(degree=(Integer(1), Integer(5))).degree() in␣
↪range(Integer(1), Integer(6)) for _ in range(Integer(10)**Integer(3)))
True
```

Note that the zero polynomial ($\pm\infty$) has degree -1. To include it, set the minimum degree to -1:

```
sage: while R.random_element(degree=(-1,2), x=-1, y=1) != R.zero():
....:     pass
```

```
>>> from sage.all import *
>>> while R.random_element(degree=(-Integer(1),Integer(2)), x=-Integer(1),␣
↪y=Integer(1)) != R.zero():
...     pass
```

Monic polynomials are chosen among all monic polynomials with degree between the given `degree` argument:

```
sage: all(R.random_element(degree=(-1, 2), monic=True).is_monic() for _ in␣
↪range(10^3))
True
```

```
>>> from sage.all import *
>>> all(R.random_element(degree=(-Integer(1), Integer(2)), monic=True).is_
↪monic() for _ in range(Integer(10)**Integer(3)))
True
```

zero()

Return the additive identity of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R = PolynomialRing(T, 'x')
sage: R.zero()
+infinity
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, 'x')
>>> R.zero()
+infinity
```

CHAPTER

FOUR

MULTIVARIATE TROPICAL POLYNOMIALS

AUTHORS:

- Verrel Rievaldo Wijaya (2024-06): initial version

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=True)
sage: R.<x,y,z> = PolynomialRing(T)
sage: z.parent()
Multivariate Tropical Polynomial Semiring in x, y, z over Rational Field
sage: R(2)*x + R(-1)*x + R(5)*y + R(-3)
(-1)*x + 5*y + (-3)
sage: (x+y+z)^2
0*x^2 + 0*x*y + 0*y^2 + 0*x*z + 0*y*z + 0*z^2
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=True)
>>> R = PolynomialRing(T, names=('x', 'y', 'z',)); (x, y, z,) = R._first_ngens(3)
>>> z.parent()
Multivariate Tropical Polynomial Semiring in x, y, z over Rational Field
>>> R(Integer(2))*x + R(-Integer(1))*x + R(Integer(5))*y + R(-Integer(3))
(-1)*x + 5*y + (-3)
>>> (x+y+z)**Integer(2)
0*x^2 + 0*x*y + 0*y^2 + 0*x*z + 0*y*z + 0*z^2
```

REFERENCES:

- [Bru2014]
- [Fil2017]

class sage.rings.semirings.tropical_mpolynomial.**TropicalMPolynomial**(*parent*, *x*)

Bases: MPolynomial_polydict

A multivariate tropical polynomial.

Let $x_1, x_2, \ldots, x_n$ be indeterminants. A tropical monomial is any product of these variables, possibly including repetitions: $x_1^{i_1} \cdots x_n^{i_n}$ where $i_j \in \{0, 1, \ldots\}$, for all $j \in \{1, \ldots, n\}$. A multivariate tropical polynomial is a finite linear combination of tropical monomials, $p(x_1, \ldots, x_n) = \sum_{i=1}^{n} c_i x_1^{i_1} \cdots x_n^{i_n}$.

In classical arithmetic, we can rewrite the general form of a tropical monomial: $x_1^{i_1} \cdots x_n^{i_n} \mapsto i_1 x_1 + \cdots + i_n x_n$. Thus, the tropical polynomial can be viewed as the minimum (maximum) of a finite collection of linear functions.

EXAMPLES:

Construct a multivariate tropical polynomial semiring in two variables:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<a,b> = PolynomialRing(T); R
Multivariate Tropical Polynomial Semiring in a, b over Rational Field
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('a', 'b',)); (a, b,) = R._first_ngens(2); R
Multivariate Tropical Polynomial Semiring in a, b over Rational Field
```

Define some multivariate tropical polynomials:

```
sage: p1 = R(3)*a*b + a + R(-1)*b; p1
3*a*b + 0*a + (-1)*b
sage: p2 = R(1)*a + R(1)*b + R(1)*a*b; p2
1*a*b + 1*a + 1*b
```

```
>>> from sage.all import *
>>> p1 = R(Integer(3))*a*b + a + R(-Integer(1))*b; p1
3*a*b + 0*a + (-1)*b
>>> p2 = R(Integer(1))*a + R(Integer(1))*b + R(Integer(1))*a*b; p2
1*a*b + 1*a + 1*b
```

Some basic arithmetic operations for multivariate tropical polynomials:

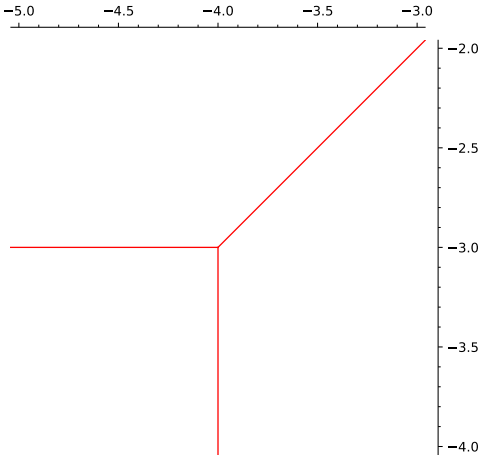
```
sage: p1 + p2
3*a*b + 1*a + 1*b
sage: p1 * p2
4*a^2*b^2 + 4*a^2*b + 4*a*b^2 + 1*a^2 + 1*a*b + 0*b^2
sage: T(2) * p1
5*a*b + 2*a + 1*b
sage: p1(T(1),T(2))
6
```

```
>>> from sage.all import *
>>> p1 + p2
3*a*b + 1*a + 1*b
>>> p1 * p2
4*a^2*b^2 + 4*a^2*b + 4*a*b^2 + 1*a^2 + 1*a*b + 0*b^2
>>> T(Integer(2)) * p1
5*a*b + 2*a + 1*b
>>> p1(T(Integer(1)),T(Integer(2)))
6
```

Let us look at the different result for tropical curve and 3d graph of tropical polynomial in two variables when different algebra is used. First for the min-plus algebra:

```
sage: T = TropicalSemiring(QQ, use_min=True)
sage: R.<a,b> = PolynomialRing(T)
sage: p1 = R(3)*a*b + a + R(-1)*b
sage: p1.tropical_variety()
Tropical curve of 3*a*b + 0*a + (-1)*b
sage: p1.tropical_variety().plot()
Graphics object consisting of 3 graphics primitives
```

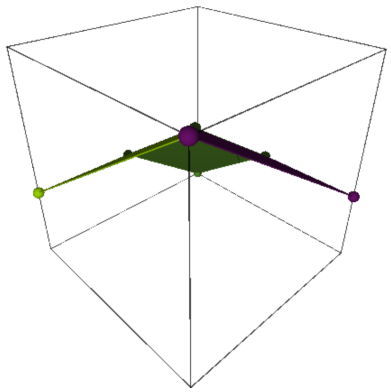
```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=True)
>>> R = PolynomialRing(T, names=('a', 'b',)); (a, b,) = R._first_ngens(2)
>>> p1 = R(Integer(3))*a*b + a + R(-Integer(1))*b
>>> p1.tropical_variety()
Tropical curve of 3*a*b + 0*a + (-1)*b
>>> p1.tropical_variety().plot()
Graphics object consisting of 3 graphics primitives
```



Tropical polynomial in two variables will induce a function in three dimension that consists of a number of surfaces:

```
sage: p1.plot3d()
Graphics3d Object
```

```
>>> from sage.all import *
>>> p1.plot3d()
Graphics3d Object
```



If we use a max-plus algebra, we will get a slightly different result:

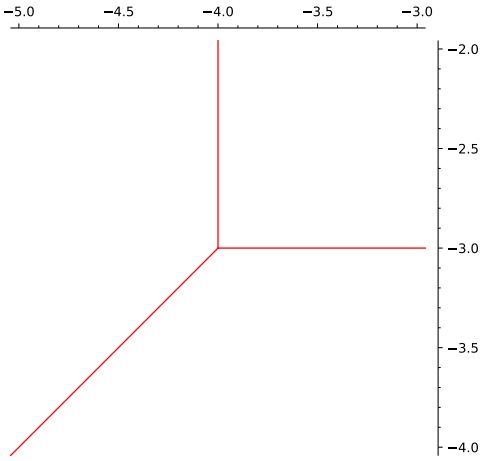
```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<a,b> = PolynomialRing(T)
```

(continues on next page)

(continued from previous page)

```
sage: p1 = R(3)*a*b + a + R(-1)*b
sage: p1.tropical_variety()
Tropical curve of 3*a*b + 0*a + (-1)*b
sage: p1.tropical_variety().plot()
Graphics object consisting of 3 graphics primitives
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('a', 'b',)); (a, b,) = R._first_ngens(2)
>>> p1 = R(Integer(3))*a*b + a + R(-Integer(1))*b
>>> p1.tropical_variety()
Tropical curve of 3*a*b + 0*a + (-1)*b
>>> p1.tropical_variety().plot()
Graphics object consisting of 3 graphics primitives
```



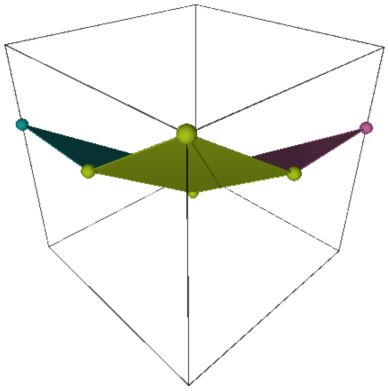

```
sage: p1.plot3d()
Graphics3d Object
```

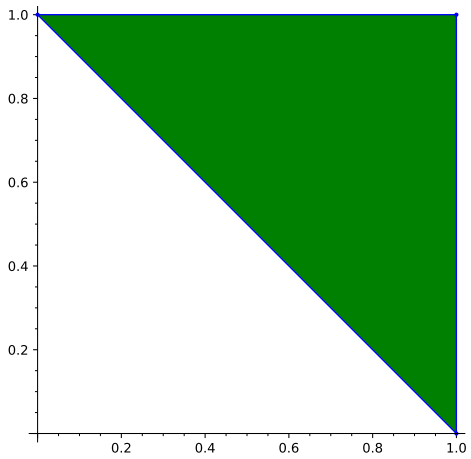
```
>>> from sage.all import *
>>> p1.plot3d()
Graphics3d Object
```

Another way to represent tropical curve is through dual subdivision, which is a subdivision of Newton polytope of tropical polynomial:

```
sage: p1.newton_polytope()
A 2-dimensional polyhedron in ZZ^2 defined as the convex hull of 3 vertices
sage: p1.dual_subdivision()
Polyhedral complex with 1 maximal cell
```

```
>>> from sage.all import *
>>> p1.newton_polytope()
A 2-dimensional polyhedron in ZZ^2 defined as the convex hull of 3 vertices
>>> p1.dual_subdivision()
Polyhedral complex with 1 maximal cell
```




1.0
0.8
0.6
0.4
0.2
0.2
0.4
0.6
0.8
1.0

dual_subdivision()

Return the dual subdivision of `self`.

Dual subdivision refers to a specific decomposition of the Newton polytope of a tropical polynomial. The term "dual" is used in the sense that the combinatorial structure of the tropical variety is reflected in the dual subdivision. Specifically, vertices of the dual subdivision correspond to the intersection of multiple components. Edges of the dual subdivision correspond to the individual components.

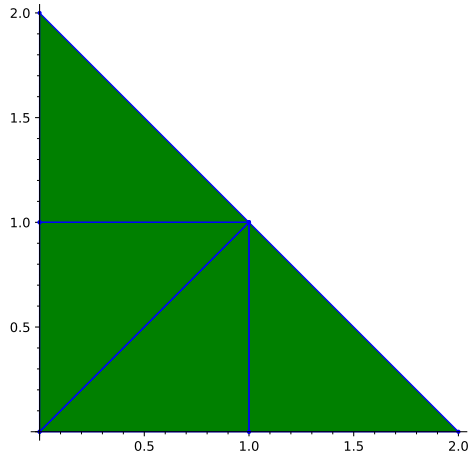
OUTPUT: `PolyhedralComplex`

EXAMPLES:

Dual subdivision of a tropical curve:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = R(3) + R(2)*x + R(2)*y + R(3)*x*y + x^2 + y^2
sage: pc = p1.dual_subdivision(); pc
Polyhedral complex with 4 maximal cells
sage: [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0), A vertex at (0, 1), A vertex at (1, 1)),
 (A vertex at (0, 0), A vertex at (1, 0), A vertex at (1, 1)),
 (A vertex at (0, 1), A vertex at (0, 2), A vertex at (1, 1)),
 (A vertex at (1, 0), A vertex at (1, 1), A vertex at (2, 0))]
```

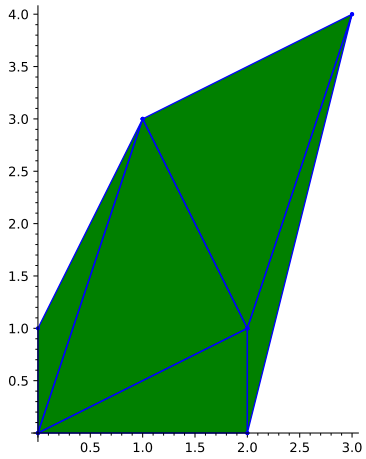
```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = R(Integer(3)) + R(Integer(2))*x + R(Integer(2))*y +␣
↪R(Integer(3))*x*y + x**Integer(2) + y**Integer(2)
>>> pc = p1.dual_subdivision(); pc
Polyhedral complex with 4 maximal cells
>>> [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0), A vertex at (0, 1), A vertex at (1, 1)),
 (A vertex at (0, 0), A vertex at (1, 0), A vertex at (1, 1)),
 (A vertex at (0, 1), A vertex at (0, 2), A vertex at (1, 1)),
 (A vertex at (1, 0), A vertex at (1, 1), A vertex at (2, 0))]
```



A subdivision of a pentagonal Newton polytope:

```
sage: p2 = R(3) + x^2 + R(-2)*y + R(1/2)*x^2*y + R(2)*x*y^3 + R(-1)*x^3*y^4
sage: pc = p2.dual_subdivision(); pc
Polyhedral complex with 5 maximal cells
sage: [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0), A vertex at (0, 1), A vertex at (1, 3)),
 (A vertex at (0, 0), A vertex at (1, 3), A vertex at (2, 1)),
 (A vertex at (0, 0), A vertex at (2, 0), A vertex at (2, 1)),
 (A vertex at (1, 3), A vertex at (2, 1), A vertex at (3, 4)),
 (A vertex at (2, 0), A vertex at (2, 1), A vertex at (3, 4))]
```

```
>>> from sage.all import *
>>> p2 = R(Integer(3)) + x**Integer(2) + R(-Integer(2))*y + R(Integer(1)/
↪Integer(2))*x**Integer(2)*y + R(Integer(2))*x*y**Integer(3) + R(-
↪Integer(1))*x**Integer(3)*y**Integer(4)
>>> pc = p2.dual_subdivision(); pc
Polyhedral complex with 5 maximal cells
>>> [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0), A vertex at (0, 1), A vertex at (1, 3)),
 (A vertex at (0, 0), A vertex at (1, 3), A vertex at (2, 1)),
 (A vertex at (0, 0), A vertex at (2, 0), A vertex at (2, 1)),
 (A vertex at (1, 3), A vertex at (2, 1), A vertex at (3, 4)),
 (A vertex at (2, 0), A vertex at (2, 1), A vertex at (3, 4))]
```



A subdivision with many faces, not all of which are triangles:

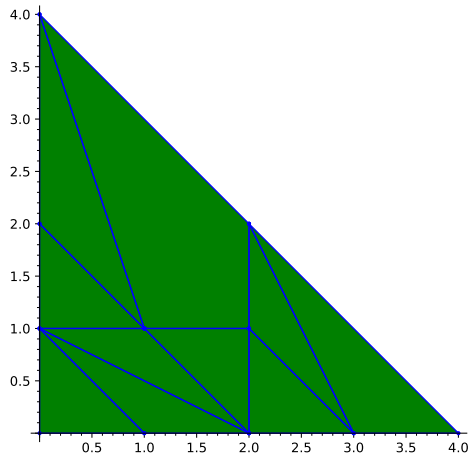
```
sage: # long time (:issue:`39569`)
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p3 = (R(8) + R(4)*x + R(2)*y + R(1)*x^2 + x*y + R(1)*y^2
....:       + R(2)*x^3 + x^2*y + x*y^2 + R(4)*y^3 + R(8)*x^4
....:       + R(4)*x^3*y + x^2*y^2 + R(2)*x*y^3 + y^4)
sage: pc = p3.dual_subdivision(); pc
Polyhedral complex with 10 maximal cells
sage: [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0), A vertex at (0, 1), A vertex at (1, 0)),
 (A vertex at (0, 1), A vertex at (0, 2), A vertex at (1, 1)),
 (A vertex at (0, 1), A vertex at (1, 0), A vertex at (2, 0)),
```

(continues on next page)

(continued from previous page)

```
 (A vertex at (0, 1), A vertex at (1, 1), A vertex at (2, 0)),
 (A vertex at (0, 2), A vertex at (0, 4), A vertex at (1, 1)),
 (A vertex at (0, 4),
  A vertex at (1, 1),
  A vertex at (2, 1),
  A vertex at (2, 2)),
 (A vertex at (1, 1), A vertex at (2, 0), A vertex at (2, 1)),
 (A vertex at (2, 0), A vertex at (2, 1), A vertex at (3, 0)),
 (A vertex at (2, 1), A vertex at (2, 2), A vertex at (3, 0)),
 (A vertex at (2, 2), A vertex at (3, 0), A vertex at (4, 0))]
```

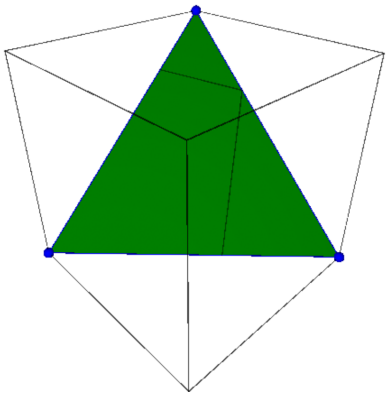
```
>>> from sage.all import *
>>> # long time (:issue:`39569`)
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p3 = (R(Integer(8)) + R(Integer(4))*x + R(Integer(2))*y +␣
↪R(Integer(1))*x**Integer(2) + x*y + R(Integer(1))*y**Integer(2)
...       + R(Integer(2))*x**Integer(3) + x**Integer(2)*y + x*y**Integer(2) +␣
↪R(Integer(4))*y**Integer(3) + R(Integer(8))*x**Integer(4)
...       + R(Integer(4))*x**Integer(3)*y + x**Integer(2)*y**Integer(2) +␣
↪R(Integer(2))*x*y**Integer(3) + y**Integer(4))
>>> pc = p3.dual_subdivision(); pc
Polyhedral complex with 10 maximal cells
>>> [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0), A vertex at (0, 1), A vertex at (1, 0)),
 (A vertex at (0, 1), A vertex at (0, 2), A vertex at (1, 1)),
 (A vertex at (0, 1), A vertex at (1, 0), A vertex at (2, 0)),
 (A vertex at (0, 1), A vertex at (1, 1), A vertex at (2, 0)),
 (A vertex at (0, 2), A vertex at (0, 4), A vertex at (1, 1)),
 (A vertex at (0, 4),
  A vertex at (1, 1),
  A vertex at (2, 1),
  A vertex at (2, 2)),
 (A vertex at (1, 1), A vertex at (2, 0), A vertex at (2, 1)),
 (A vertex at (2, 0), A vertex at (2, 1), A vertex at (3, 0)),
 (A vertex at (2, 1), A vertex at (2, 2), A vertex at (3, 0)),
 (A vertex at (2, 2), A vertex at (3, 0), A vertex at (4, 0))]
```



Dual subdivision of a tropical surface:

```
sage: # long time (:issue:`39569`)
sage: T = TropicalSemiring(QQ)
sage: R.<x,y,z> = PolynomialRing(T)
sage: p1 = x + y + z + x^2 + R(1)
sage: pc = p1.dual_subdivision(); pc
Polyhedral complex with 7 maximal cells
sage: [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0, 0), A vertex at (0, 0, 1), A vertex at (0, 1, 0)),
 (A vertex at (0, 0, 0), A vertex at (0, 0, 1), A vertex at (1, 0, 0)),
 (A vertex at (0, 0, 0), A vertex at (0, 1, 0), A vertex at (1, 0, 0)),
 (A vertex at (0, 0, 1), A vertex at (0, 1, 0), A vertex at (1, 0, 0)),
 (A vertex at (0, 0, 1), A vertex at (0, 1, 0), A vertex at (2, 0, 0)),
 (A vertex at (0, 0, 1), A vertex at (1, 0, 0), A vertex at (2, 0, 0)),
 (A vertex at (0, 1, 0), A vertex at (1, 0, 0), A vertex at (2, 0, 0))]
```

```
>>> from sage.all import *
>>> # long time (:issue:`39569`)
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y', 'z',)); (x, y, z,) = R._first_
↪ngens(3)
>>> p1 = x + y + z + x**Integer(2) + R(Integer(1))
>>> pc = p1.dual_subdivision(); pc
Polyhedral complex with 7 maximal cells
>>> [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0, 0), A vertex at (0, 0, 1), A vertex at (0, 1, 0)),
 (A vertex at (0, 0, 0), A vertex at (0, 0, 1), A vertex at (1, 0, 0)),
 (A vertex at (0, 0, 0), A vertex at (0, 1, 0), A vertex at (1, 0, 0)),
 (A vertex at (0, 0, 1), A vertex at (0, 1, 0), A vertex at (1, 0, 0)),
 (A vertex at (0, 0, 1), A vertex at (0, 1, 0), A vertex at (2, 0, 0)),
 (A vertex at (0, 0, 1), A vertex at (1, 0, 0), A vertex at (2, 0, 0)),
 (A vertex at (0, 1, 0), A vertex at (1, 0, 0), A vertex at (2, 0, 0))]
```



Dual subdivision of a tropical hypersurface:

```
sage: T = TropicalSemiring(QQ)
sage: R.<a,b,c,d> = PolynomialRing(T)
```

(continues on next page)

(continued from previous page)

```
sage: p1 = R(2)*a*b + R(3)*a*c + R(-1)*c^2 + R(-1/3)*a*d
sage: pc = p1.dual_subdivision(); pc
Polyhedral complex with 4 maximal cells
sage: [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0, 2, 0),
 A vertex at (1, 0, 0, 1),
 A vertex at (1, 0, 1, 0)),
(A vertex at (0, 0, 2, 0),
 A vertex at (1, 0, 0, 1),
 A vertex at (1, 1, 0, 0)),
(A vertex at (0, 0, 2, 0),
 A vertex at (1, 0, 1, 0),
 A vertex at (1, 1, 0, 0)),
(A vertex at (1, 0, 0, 1),
 A vertex at (1, 0, 1, 0),
 A vertex at (1, 1, 0, 0))]
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('a', 'b', 'c', 'd',)); (a, b, c, d,) = R._
↪first_ngens(4)
>>> p1 = R(Integer(2))*a*b + R(Integer(3))*a*c + R(-Integer(1))*c**Integer(2)␣
↪+ R(-Integer(1)/Integer(3))*a*d
>>> pc = p1.dual_subdivision(); pc
Polyhedral complex with 4 maximal cells
>>> [p.Vrepresentation() for p in pc.maximal_cells_sorted()]
[(A vertex at (0, 0, 2, 0),
 A vertex at (1, 0, 0, 1),
 A vertex at (1, 0, 1, 0)),
(A vertex at (0, 0, 2, 0),
 A vertex at (1, 0, 0, 1),
 A vertex at (1, 1, 0, 0)),
(A vertex at (0, 0, 2, 0),
 A vertex at (1, 0, 1, 0),
 A vertex at (1, 1, 0, 0)),
(A vertex at (1, 0, 0, 1),
 A vertex at (1, 0, 1, 0),
 A vertex at (1, 1, 0, 0))]
```

newton_polytope()

Return the Newton polytope of `self`.

The Newton polytope is the convex hull of all the points corresponding to the exponents of the monomials of tropical polynomial.

OUTPUT: `Polyhedron()`

EXAMPLES:

A Newton polytope for a two-variable tropical polynomial:

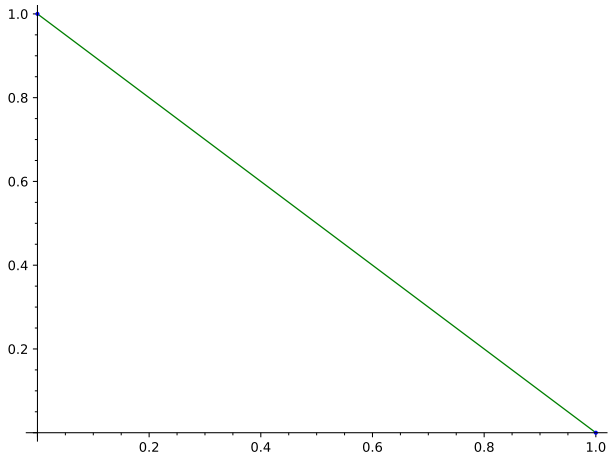
```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = x + y
```

(continues on next page)

(continued from previous page)

```
sage: p1.newton_polytope()
A 1-dimensional polyhedron in ZZ^2 defined as the convex hull of 2 vertices
sage: p1.newton_polytope().Vrepresentation()
(A vertex at (0, 1), A vertex at (1, 0))
sage: p1.newton_polytope().Hrepresentation()
(An equation (1, 1) x - 1 == 0,
 An inequality (0, -1) x + 1 >= 0,
 An inequality (0, 1) x + 0 >= 0)
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = x + y
>>> p1.newton_polytope()
A 1-dimensional polyhedron in ZZ^2 defined as the convex hull of 2 vertices
>>> p1.newton_polytope().Vrepresentation()
(A vertex at (0, 1), A vertex at (1, 0))
>>> p1.newton_polytope().Hrepresentation()
(An equation (1, 1) x - 1 == 0,
 An inequality (0, -1) x + 1 >= 0,
 An inequality (0, 1) x + 0 >= 0)
```



A Newton polytope in three dimension:

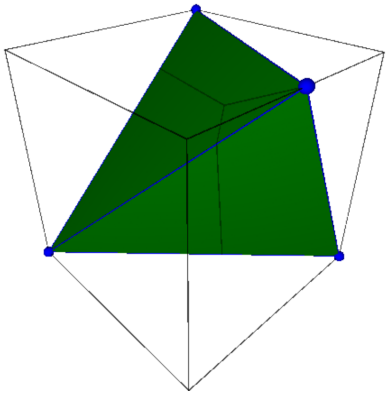
```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y,z> = PolynomialRing(T)
sage: p1 = x^2 + x*y*z + x + y + z + R(0)
sage: p1.newton_polytope()
A 3-dimensional polyhedron in ZZ^3 defined as the convex hull of 5 vertices
sage: p1.newton_polytope().Vrepresentation()
(A vertex at (0, 0, 0),
A vertex at (0, 0, 1),
A vertex at (0, 1, 0),
A vertex at (2, 0, 0),
A vertex at (1, 1, 1))
sage: p1.newton_polytope().Hrepresentation()
(An inequality (0, 1, 0) x + 0 >= 0,
```

(continues on next page)

(continued from previous page)

```
 An inequality (0, 0, 1) x + 0 >= 0,
 An inequality (1, 0, 0) x + 0 >= 0,
 An inequality (1, -1, -1) x + 1 >= 0,
 An inequality (-1, -2, 1) x + 2 >= 0,
 An inequality (-1, 1, -2) x + 2 >= 0)
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y', 'z',)); (x, y, z,) = R._first_
↪ngens(3)
>>> p1 = x**Integer(2) + x*y*z + x + y + z + R(Integer(0))
>>> p1.newton_polytope()
A 3-dimensional polyhedron in ZZ^3 defined as the convex hull of 5 vertices
>>> p1.newton_polytope().Vrepresentation()
(A vertex at (0, 0, 0),
A vertex at (0, 0, 1),
A vertex at (0, 1, 0),
A vertex at (2, 0, 0),
A vertex at (1, 1, 1))
>>> p1.newton_polytope().Hrepresentation()
(An inequality (0, 1, 0) x + 0 >= 0,
 An inequality (0, 0, 1) x + 0 >= 0,
 An inequality (1, 0, 0) x + 0 >= 0,
 An inequality (1, -1, -1) x + 1 >= 0,
 An inequality (-1, -2, 1) x + 2 >= 0,
 An inequality (-1, 1, -2) x + 2 >= 0)
```



plot3d(*color='random'*)

Return the 3d plot of `self`.

Only implemented for tropical polynomial in two variables. The x-y axes for this 3d plot is the same as the x-y axes of the corresponding tropical curve.

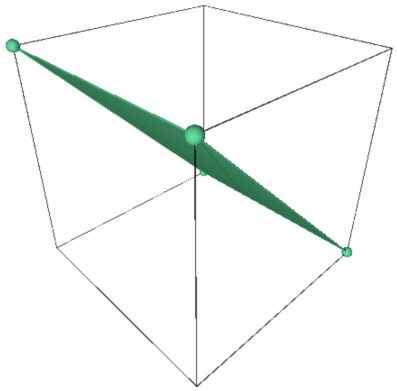
OUTPUT: Graphics3d Object

EXAMPLES:

A simple tropical polynomial that consist of only one surface:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = x^2
sage: p1.plot3d()
Graphics3d Object
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = x**Integer(2)
>>> p1.plot3d()
Graphics3d Object
```



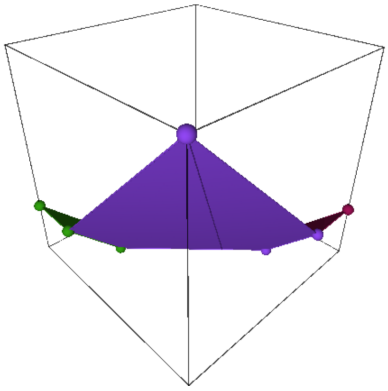
Tropical polynomials often have graphs that represent a combination of multiple surfaces:

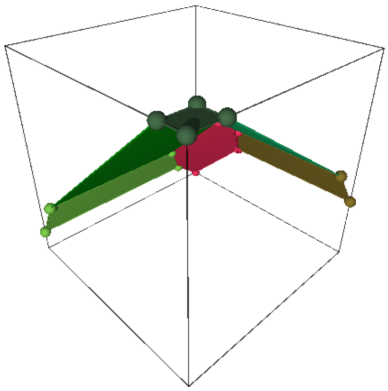
```
sage: p2 = R(3) + R(2)*x + R(2)*y + R(3)*x*y
sage: p2.plot3d()
Graphics3d Object
```

```
>>> from sage.all import *
>>> p2 = R(Integer(3)) + R(Integer(2))*x + R(Integer(2))*y + R(Integer(3))*x*y
>>> p2.plot3d()
Graphics3d Object
```

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p3 = R(2)*x^2 + x*y + R(2)*y^2 + x + R(-1)*y + R(3)
sage: p3.plot3d()
Graphics3d Object
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p3 = R(Integer(2))*x**Integer(2) + x*y + R(Integer(2))*y**Integer(2) + x␣
↪+ R(-Integer(1))*y + R(Integer(3))
>>> p3.plot3d()
Graphics3d Object
```





subs(*fixed=None*, ***kwds*)

Fix some given variables in `self` and return the changed tropical multivariate polynomials.

See also

`sage.rings.polynomial.multi_polynomial_element.MPolynomial_polydict.subs()`

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = x^2 + y + R(3)
sage: p1((R(4),y))
0*y + 8
sage: p1.subs({x: 4})
0*y + 8
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = x**Integer(2) + y + R(Integer(3))
>>> p1((R(Integer(4)),y))
0*y + 8
>>> p1.subs({x: Integer(4)})
0*y + 8
```

tropical_variety()

Return tropical roots of `self`.

In the multivariate case, the roots can be represented by a tropical variety. In two dimensions, this is known as a tropical curve. For dimensions higher than two, it is referred to as a tropical hypersurface.

OUTPUT: *sage.rings.semirings.tropical_variety.TropicalVariety*

EXAMPLES:

Tropical curve for tropical polynomials in two variables:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = x + y + R(0); p1
0*x + 0*y + 0
sage: p1.tropical_variety()
Tropical curve of 0*x + 0*y + 0
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = x + y + R(Integer(0)); p1
0*x + 0*y + 0
>>> p1.tropical_variety()
Tropical curve of 0*x + 0*y + 0
```

Tropical hypersurface for tropical polynomials in more than two variables:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y,z> = PolynomialRing(T)
sage: p1 = R(1)*x*y + R(-1/2)*x*z + R(4)*z^2; p1
1*x*y + (-1/2)*x*z + 4*z^2
sage: p1.tropical_variety()
Tropical surface of 1*x*y + (-1/2)*x*z + 4*z^2
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y', 'z',)); (x, y, z,) = R._first_
↪ngens(3)
>>> p1 = R(Integer(1))*x*y + R(-Integer(1)/Integer(2))*x*z +␣
↪R(Integer(4))*z**Integer(2); p1
1*x*y + (-1/2)*x*z + 4*z^2
>>> p1.tropical_variety()
Tropical surface of 1*x*y + (-1/2)*x*z + 4*z^2
```

class sage.rings.semirings.tropical_mpolynomial.**TropicalMPolynomialSemiring**(*base_semiring*, *n*, *names*, *order*)

Bases: UniqueRepresentation, Parent

The semiring of tropical polynomials in multiple variables.

This is the commutative semiring consisting of all finite linear combinations of tropical monomials under (tropical) addition and multiplication with coefficients in a tropical semiring.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: f = T(1)*x + T(-1)*y
sage: g = T(2)*x + T(-2)*y
sage: f + g
1*x + (-2)*y
sage: f * g
3*x^2 + (-1)*x*y + (-3)*y^2
sage: f + R.zero() == f
True
sage: f * R.zero() == R.zero()
True
sage: f * R.one() == f
True
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> f = T(Integer(1))*x + T(-Integer(1))*y
>>> g = T(Integer(2))*x + T(-Integer(2))*y
>>> f + g
1*x + (-2)*y
>>> f * g
3*x^2 + (-1)*x*y + (-3)*y^2
```

(continues on next page)

(continued from previous page)

```
>>> f + R.zero() == f
True
>>> f * R.zero() == R.zero()
True
>>> f * R.one() == f
True
```

Element

alias of *TropicalMPolynomial*

gen(*n=0*)

Return the `n`-th generator of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<a,b,c> = PolynomialRing(T)
sage: R.gen()
0*a
sage: R.gen(2)
0*c
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('a', 'b', 'c',)); (a, b, c,) = R._first_
↪ngens(3)
>>> R.gen()
0*a
>>> R.gen(Integer(2))
0*c
```

gens()

Return the generators of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R = PolynomialRing(T, 5, 'x')
sage: R.gens()
(0*x0, 0*x1, 0*x2, 0*x3, 0*x4)
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, Integer(5), 'x')
>>> R.gens()
(0*x0, 0*x1, 0*x2, 0*x3, 0*x4)
```

ngens()

Return the number of generators of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R = PolynomialRing(T, 10, 'z')
sage: R.ngens()
10
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, Integer(10), 'z')
>>> R.ngens()
10
```

one()

Return the multiplicative identity of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R = PolynomialRing(T, 'x,y')
sage: R.one()
0
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, 'x,y')
>>> R.one()
0
```

random_element(*degree=2*, *terms=None*, *choose_degree=False*, **args*, ***kwargs*)

Return a random multivariate tropical polynomial from `self`.

OUTPUT: *TropicalMPolynomial*

See also

`sage.rings.polynomial.multi_polynomial_ring_base.MPolynomialRing_base.random_element()`

EXAMPLES:

A random polynomial of at most degree d and at most t terms:

```
sage: T = TropicalSemiring(QQ)
sage: R.<a,b,c> = PolynomialRing(T)
sage: f = R.random_element(2, 5)
sage: f.degree() <= 2
True
sage: f.parent() is R
True
sage: len(list(f)) <= 5
True
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('a', 'b', 'c',)); (a, b, c,) = R._first_
↪ngens(3)
>>> f = R.random_element(Integer(2), Integer(5))
>>> f.degree() <= Integer(2)
True
>>> f.parent() is R
True
>>> len(list(f)) <= Integer(5)
True
```

Choose degrees of monomials randomly first rather than monomials uniformly random:

```
sage: f = R.random_element(3, 6, choose_degree=True)
sage: f.degree() <= 3
True
sage: f.parent() is R
True
sage: len(list(f)) <= 6
True
```

```
>>> from sage.all import *
>>> f = R.random_element(Integer(3), Integer(6), choose_degree=True)
>>> f.degree() <= Integer(3)
True
>>> f.parent() is R
True
>>> len(list(f)) <= Integer(6)
True
```

term_order()

Return the defined term order of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y,z> = PolynomialRing(T)
sage: R.term_order()
Degree reverse lexicographic term order
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y', 'z',)); (x, y, z,) = R._first_
↪ngens(3)
>>> R.term_order()
Degree reverse lexicographic term order
```

zero()

Return the additive identity of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R = PolynomialRing(T, 'x,y')
sage: R.zero()
+infinity
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, 'x,y')
>>> R.zero()
+infinity
```

CHAPTER

FIVE

MATRICES OVER TROPICAL SEMIRINGS

AUTHORS:

- Xavier Caruso (2025-11): initial version

class sage.rings.semirings.tropical_matrix.**Matrix_tropical_dense**

Bases: Matrix_generic_dense

A class for dense matrices over a tropical semiring.

EXAMPLES:

```
sage: from sage.rings.semirings.tropical_matrix import Matrix_tropical_dense
sage: T = TropicalSemiring(QQ)
sage: M = matrix(T, [[1, 2], [3, 4]])
sage: isinstance(M, Matrix_tropical_dense)
True
```

```
>>> from sage.all import *
>>> from sage.rings.semirings.tropical_matrix import Matrix_tropical_dense
>>> T = TropicalSemiring(QQ)
>>> M = matrix(T, [[Integer(1), Integer(2)], [Integer(3), Integer(4)]])
>>> isinstance(M, Matrix_tropical_dense)
True
```

extremum_cycle_mean()

Return the extremal (that is, minimal if the addition is max and maximal is the addition is min) mean weight of this matrix It is also the smallest/largest eigenvalue of this matrix.

ALGORITHM:

We implement Karp's algorithm described in [But2010], Section 1.6.1.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: M = matrix(T, [[-2,  1, -3],
....:                [ 3,  0,  3],
....:                [ 5,  2,  1]])
sage: M.extremum_cycle_mean()
3
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
```

(continues on next page)

(continued from previous page)

```
>>> M = matrix(T, [[-Integer(2),  Integer(1), -Integer(3)],
...                [ Integer(3),  Integer(0),  Integer(3)],
...                [ Integer(5),  Integer(2),  Integer(1)]])
>>> M.extremum_cycle_mean()
3
```

```
sage: T = TropicalSemiring(QQ)
sage: z = T.zero()
sage: M = matrix(T, [[z, 1, 10, z],
....:                [z, z,  3, z],
....:                [z, z,  z, 2],
....:                [8, 0,  z, z]])
sage: M.extremum_cycle_mean()
5/3
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> z = T.zero()
>>> M = matrix(T, [[z, Integer(1), Integer(10), z],
...                [z, z,  Integer(3), z],
...                [z, z,  z, Integer(2)],
...                [Integer(8), Integer(0),  z, z]])
>>> M.extremum_cycle_mean()
5/3
```

kleene_star()

alias of *strong_transitive_closure()*.

strong_transitive_closure()

Return the string transitive closure of this matrix M, that is, by definition

$$I \oplus A \oplus A^2 \oplus A^3 \oplus A^4 \oplus \cdots$$

or raise an error if this sum does not converge.

ALGORITHM:

We implement the Floyd-Warshall algorithm described in [But2010], Algorithm 1.6.21.

See also

weak_transitive_closure()

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: M = matrix(T, [[-5, -2, -6],
....:                [ 0, -3,  0],
....:                [ 2, -1, -2]])
sage: M.strong_transitive_closure()
[ 0 -2 -2]
[ 2  0  0]
[ 2  0  0]
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> M = matrix(T, [[-Integer(5), -Integer(2), -Integer(6)],
...                [ Integer(0), -Integer(3),  Integer(0)],
...                [ Integer(2), -Integer(1), -Integer(2)]])
>>> M.strong_transitive_closure()
[ 0 -2 -2]
[ 2  0  0]
[ 2  0  0]
```

```
sage: T = TropicalSemiring(QQ)
sage: M = matrix(T, [[-5, -2, -6],
....:               [ 0, -3,  0],
....:               [ 2, -1, -2]])
sage: M.strong_transitive_closure()
Traceback (most recent call last):
...
ValueError: negative cycle exists
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> M = matrix(T, [[-Integer(5), -Integer(2), -Integer(6)],
...                [ Integer(0), -Integer(3),  Integer(0)],
...                [ Integer(2), -Integer(1), -Integer(2)]])
>>> M.strong_transitive_closure()
Traceback (most recent call last):
...
ValueError: negative cycle exists
```

weak_transitive_closure()

Return the weak transitive closure of this matrix M, that is, by definition

$$A \oplus A^2 \oplus A^3 \oplus A^4 \oplus \cdots$$

or raise an error if this sum does not converge.

ALGORITHM:

We implement the Floyd-Warshall algorithm described in [But2010], Algorithm 1.6.21.

See also

strong_transitive_closure()

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: z = T.zero()
sage: M = matrix(T, [[z, 1, 10, z],
....:               [z, z,  3, z],
....:               [z, z,  z, 2],
....:               [8, 0,  z, z]])
```

(continues on next page)

(continued from previous page)

```
sage: M.weak_transitive_closure()
[14  1  4  6]
[13  5  3  5]
[10  2  5  2]
[ 8  0  3  5]
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> z = T.zero()
>>> M = matrix(T, [[z, Integer(1), Integer(10), z],
...                [z, z,  Integer(3), z],
...                [z, z,  z, Integer(2)],
...                [Integer(8), Integer(0),  z, z]])
>>> M.weak_transitive_closure()
[14  1  4  6]
[13  5  3  5]
[10  2  5  2]
[ 8  0  3  5]
```

We check that the minimal cycle mean of M is the largest value a such that $(-a) \otimes M$ has a weak transitive closure:

```
sage: M.extremum_cycle_mean()
5/3
sage: aM = T(-5/3) * M
sage: aM.weak_transitive_closure()
[22/3 -2/3  2/3    1]
[   8    0  4/3  5/3]
[20/3 -4/3    0  1/3]
[19/3 -5/3 -1/3    0]
sage: bM = T(-2) * M
sage: bM.weak_transitive_closure()
Traceback (most recent call last):
...
ValueError: negative cycle exists
```

```
>>> from sage.all import *
>>> M.extremum_cycle_mean()
5/3
>>> aM = T(-Integer(5)/Integer(3)) * M
>>> aM.weak_transitive_closure()
[22/3 -2/3  2/3    1]
[   8    0  4/3  5/3]
[20/3 -4/3    0  1/3]
[19/3 -5/3 -1/3    0]
>>> bM = T(-Integer(2)) * M
>>> bM.weak_transitive_closure()
Traceback (most recent call last):
...
ValueError: negative cycle exists
```

CHAPTER

SIX

TROPICAL VARIETIES

A tropical variety is a piecewise-linear geometric object derived from a classical algebraic variety by using tropical mathematics, where the tropical semiring replaces the usual arithmetic operations.

AUTHORS:

- Verrel Rievaldo Wijaya (2024-06): initial version

REFERENCES:

- [Bru2014]
- [Mac2015]
- [Fil2017]

class sage.rings.semirings.tropical_variety.**TropicalCurve**(*poly*)

Bases: *TropicalVariety*

A tropical curve in $\mathbf{R}^2$.

The tropical curve consists of line segments and half-lines, which we call edges. These edges are connected in such a way that they form a piecewise linear graph embedded in the plane. These edges meet at a vertices, where the balancing condition is satisfied. This balancing condition ensures that the sum of the outgoing slopes at each vertex is zero, reflecting the equilibrium.

EXAMPLES:

We define some tropical curves:

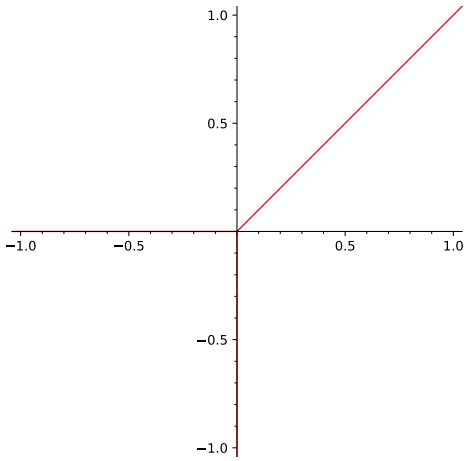
```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = x + y + R(0)
sage: tv1 = p1.tropical_variety(); tv1
Tropical curve of 0*x + 0*y + 0
sage: tv1.components()
[[(t1, t1), [t1 >= 0], 1], [(0, t1), [t1 <= 0], 1], [(t1, 0), [t1 <= 0], 1]]
sage: tv1.plot()
Graphics object consisting of 3 graphics primitives
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = x + y + R(Integer(0))
>>> tv1 = p1.tropical_variety(); tv1
Tropical curve of 0*x + 0*y + 0
```

(continues on next page)

(continued from previous page)

```
>>> tv1.components()
[[(t1, t1), [t1 >= 0], 1], [(0, t1), [t1 <= 0], 1], [(t1, 0), [t1 <= 0], 1]]
>>> tv1.plot()
Graphics object consisting of 3 graphics primitives
```




```
sage: p2 = R(-2)*x^2 + R(-1)*x + R(1/2)*y + R(1/6)
sage: tv2 = p2.tropical_variety()
sage: tv2.components()
[[(1/2*t1 + 5/4, t1), [(-1/3) <= t1], 1],
 [(13/12, t1), [t1 <= (-1/3)], 2],
 [(t1, -1/3), [t1 <= (13/12)], 1]]
sage: tv2.plot()
Graphics object consisting of 4 graphics primitives
```

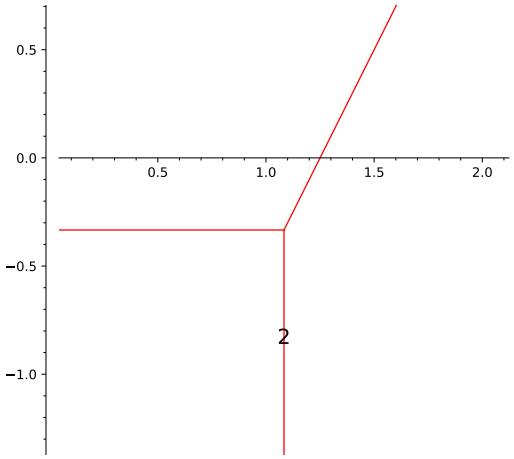
```
>>> from sage.all import *
>>> p2 = R(-Integer(2))*x**Integer(2) + R(-Integer(1))*x + R(Integer(1)/
↪Integer(2))*y + R(Integer(1)/Integer(6))
>>> tv2 = p2.tropical_variety()
>>> tv2.components()
[[(1/2*t1 + 5/4, t1), [(-1/3) <= t1], 1],
 [(13/12, t1), [t1 <= (-1/3)], 2],
 [(t1, -1/3), [t1 <= (13/12)], 1]]
>>> tv2.plot()
Graphics object consisting of 4 graphics primitives
```

When two tropical polynomials are multiplied, the tropical curve of the resulting polynomial is the union of the tropical curves of the original polynomials:

```
sage: p3 = p1 * p2; p3
(-2)*x^3 + (-2)*x^2*y + (-1)*x^2 + 1/2*x*y + 1/2*y^2 + 1/6*x + 1/2*y + 1/6
sage: tv3 = p3.tropical_variety()
sage: tv3.plot()
Graphics object consisting of 11 graphics primitives
```

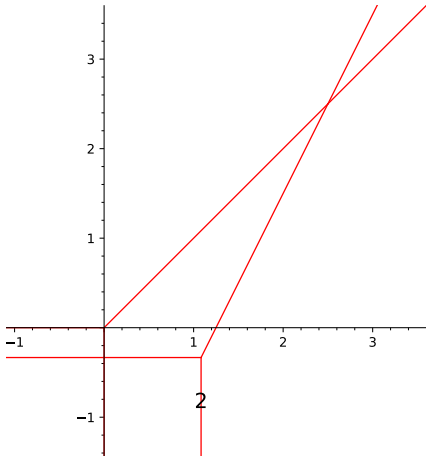
```
>>> from sage.all import *
>>> p3 = p1 * p2; p3
```

(continues on next page)



(continued from previous page)

```
(-2)*x^3 + (-2)*x^2*y + (-1)*x^2 + 1/2*x*y + 1/2*y^2 + 1/6*x + 1/2*y + 1/6
>>> tv3 = p3.tropical_variety()
>>> tv3.plot()
Graphics object consisting of 11 graphics primitives
```



contribution()

Return the contribution of `self`.

The contribution of a simple curve C is defined as the product of the normalized areas of all triangles in the corresponding dual subdivision. We just multiply positive integers attached to the trivalent vertices. The contribution of a trivalent vertex equals $w_1 w_2 |\det(v_1, v_2)|$, with w_i are the weights of the adjacent edges and v_i are their weight vectors. That formula is independent of the choice made because of the balancing condition $w_1 v_1 + w_2 v_2 + w_3 v_3 = 0$.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = R(2)*x^2 + x*y + R(2)*y^2 + x + R(-1)*y + R(3)
sage: p1.tropical_variety().contribution()
1
sage: p2 = R(-1/3)*x^2 + R(1)*x*y + R(1)*y^2 + R(-1/3)*x + R(1/3)
```

(continues on next page)

(continued from previous page)

```
sage: p2.tropical_variety().contribution()
16
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = R(Integer(2))*x**Integer(2) + x*y + R(Integer(2))*y**Integer(2) + x␣
↪+ R(-Integer(1))*y + R(Integer(3))
>>> p1.tropical_variety().contribution()
1
>>> p2 = R(-Integer(1)/Integer(3))*x**Integer(2) + R(Integer(1))*x*y +␣
↪R(Integer(1))*y**Integer(2) + R(-Integer(1)/Integer(3))*x + R(Integer(1)/
↪Integer(3))
>>> p2.tropical_variety().contribution()
16
```

genus()

Return the genus of `self`.

Let $t(C)$ be the number of trivalent vertices, and let $r(C)$ be the number of unbounded edges of C. The genus of simple tropical curve C is defined by the formula:

$$g(C) = \frac{1}{2}t(C) - \frac{1}{2}r(C) + 1.$$

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = x^2 + y^2 + x*y
sage: p1.tropical_variety().genus()
1
sage: p2 = R(2)*x^2 + x*y + R(2)*y^2 + x + R(-1)*y + R(3)
sage: p2.tropical_variety().genus()
0
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = x**Integer(2) + y**Integer(2) + x*y
>>> p1.tropical_variety().genus()
1
>>> p2 = R(Integer(2))*x**Integer(2) + x*y + R(Integer(2))*y**Integer(2) + x␣
↪+ R(-Integer(1))*y + R(Integer(3))
>>> p2.tropical_variety().genus()
0
```

is_simple()

Return `True` if `self` is simple and `False` otherwise.

A tropical curve C is called simple if each vertex is either trivalent or is locally the intersection of two line segments. Equivalently, C is simple if the corresponding subdivision consists only of triangles and parallelograms.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = R(0) + x + y + x*y + x^2*y + x*y^2
sage: p1.tropical_variety().is_simple()
False
sage: p2 = R(2)*x^2 + x*y + R(2)*y^2 + x + R(-1)*y + R(3)
sage: p2.tropical_variety().is_simple()
True
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = R(Integer(0)) + x + y + x*y + x**Integer(2)*y + x*y**Integer(2)
>>> p1.tropical_variety().is_simple()
False
>>> p2 = R(Integer(2))*x**Integer(2) + x*y + R(Integer(2))*y**Integer(2) + x␣
↪+ R(-Integer(1))*y + R(Integer(3))
>>> p2.tropical_variety().is_simple()
True
```

is_smooth()

Return `True` if `self` is smooth and `False` otherwise.

Suppose C is a tropical curve of degree d. A tropical curve C is smooth if the dual subdivision of C consists of d^2 triangles each having unit area $1/2$. This is equivalent with C having d^2 vertices. These vertices are necessarily trivalent (has three adjacent edges).

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = x^2 + x + R(1)
sage: p1.tropical_variety().is_smooth()
False
sage: p2 = R(2)*x^2 + x*y + R(2)*y^2 + x + R(-1)*y + R(3)
sage: p2.tropical_variety().is_smooth()
True
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = x**Integer(2) + x + R(Integer(1))
>>> p1.tropical_variety().is_smooth()
False
>>> p2 = R(Integer(2))*x**Integer(2) + x*y + R(Integer(2))*y**Integer(2) + x␣
↪+ R(-Integer(1))*y + R(Integer(3))
>>> p2.tropical_variety().is_smooth()
True
```

plot()

Return the plot of `self`.

Generates a visual representation of the tropical curve in cartesian coordinates. The plot shows piecewise-linear segments representing each components. The axes are centered around the vertices.

OUTPUT:

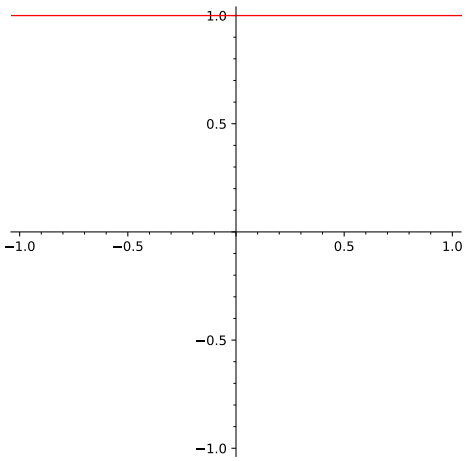
A Graphics object. The weight of the component will be written if it is greater or equal than 2. The weight is written near the vertex.

EXAMPLES:

A polynomial with only two terms will give one straight line:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: (y+R(1)).tropical_variety().components()
[[(t1, 1), [-Infinity < t1, t1 < +Infinity], 1]]
sage: (y+R(1)).tropical_variety().plot()
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> (y+R(Integer(1))).tropical_variety().components()
[[(t1, 1), [-Infinity < t1, t1 < +Infinity], 1]]
>>> (y+R(Integer(1))).tropical_variety().plot()
Graphics object consisting of 1 graphics primitive
```



An intriguing and fascinating tropical curve can be obtained with a more complex tropical polynomial:

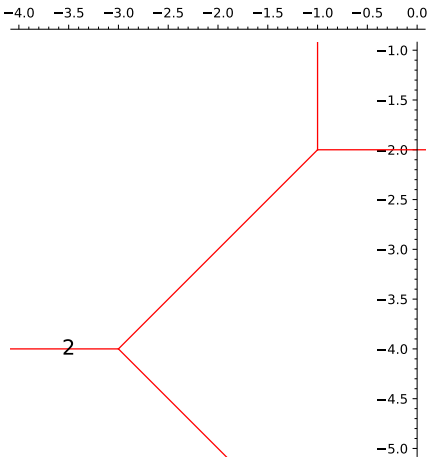
```
sage: p1 = R(1) + R(2)*x + R(3)*y + R(6)*x*y + R(10)*x*y^2
sage: p1.tropical_variety().components()
[[(-1, t1), [-2 <= t1], 1],
[(t1, -2), [-1 <= t1], 1],
[(t1 + 1, t1), [-4 <= t1, t1 <= -2], 1],
[(t1, -4), [t1 <= -3], 2],
[(-t1 - 7, t1), [t1 <= -4], 1]]
sage: p1.tropical_variety().plot()
Graphics object consisting of 6 graphics primitives
```

```
>>> from sage.all import *
>>> p1 = R(Integer(1)) + R(Integer(2))*x + R(Integer(3))*y +␣
↪R(Integer(6))*x*y + R(Integer(10))*x*y**Integer(2)
```

(continues on next page)

(continued from previous page)

```
>>> p1.tropical_variety().components()
[[(-1, t1), [-2 <= t1], 1],
[(t1, -2), [-1 <= t1], 1],
[(t1 + 1, t1), [-4 <= t1, t1 <= -2], 1],
[(t1, -4), [t1 <= -3], 2],
[(-t1 - 7, t1), [t1 <= -4], 1]]
>>> p1.tropical_variety().plot()
Graphics object consisting of 6 graphics primitives
```



Another tropical polynomial with numerous components, resulting in a more intricate structure:

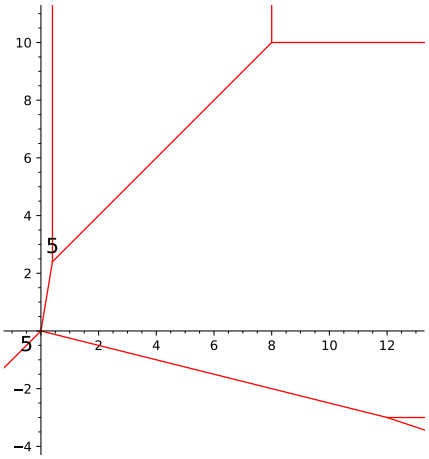
```
sage: p2 = (x^6 + R(4)*x^4*y^2 + R(2)*x^3*y^3 + R(3)*x^2*y^4
....:       + x*y^5 + R(7)*x^2 + R(5)*x*y + R(3)*y^2 + R(2)*x
....:       + y + R(10))
sage: p2.tropical_variety().plot() # long time
Graphics object consisting of 11 graphics primitives
```

```
>>> from sage.all import *
>>> p2 = (x**Integer(6) + R(Integer(4))*x**Integer(4)*y**Integer(2) +␣
↪R(Integer(2))*x**Integer(3)*y**Integer(3) +␣
↪R(Integer(3))*x**Integer(2)*y**Integer(4)
...       + x*y**Integer(5) + R(Integer(7))*x**Integer(2) + R(Integer(5))*x*y␣
↪+ R(Integer(3))*y**Integer(2) + R(Integer(2))*x
...       + y + R(Integer(10)))
>>> p2.tropical_variety().plot() # long time
Graphics object consisting of 11 graphics primitives
```

```
sage: p3 = (R(8) + R(4)*x + R(2)*y + R(1)*x^2 + x*y + R(1)*y^2
....:       + R(2)*x^3 + x^2*y + x*y^2 + R(4)*y^3 + R(8)*x^4
....:       + R(4)*x^3*y + x^2*y^2 + R(2)*x*y^3 + y^4)
sage: p3.tropical_variety().plot() # long time
Graphics object consisting of 23 graphics primitives
```

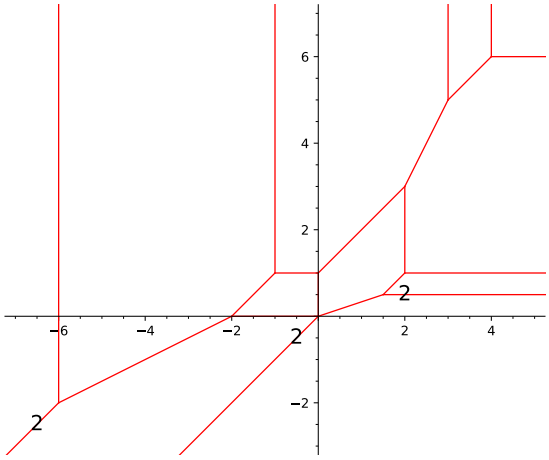
```
>>> from sage.all import *
>>> p3 = (R(Integer(8)) + R(Integer(4))*x + R(Integer(2))*y +␣
↪R(Integer(1))*x**Integer(2) + x*y + R(Integer(1))*y**Integer(2)
```

(continues on next page)



(continued from previous page)

```
...       + R(Integer(2))*x**Integer(3) + x**Integer(2)*y + x*y**Integer(2) +␣
↪R(Integer(4))*y**Integer(3) + R(Integer(8))*x**Integer(4)
...       + R(Integer(4))*x**Integer(3)*y + x**Integer(2)*y**Integer(2) +␣
↪R(Integer(2))*x*y**Integer(3) + y**Integer(4))
>>> p3.tropical_variety().plot() # long time
Graphics object consisting of 23 graphics primitives
```



vertices()

Return all vertices of `self`, which is the point where three or more edges intersect.

OUTPUT: a set of (x, y) points

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = x + y
sage: p1.tropical_variety().vertices()
set()
sage: p2 = R(-2)*x^2 + R(-1)*x + R(1/2)*y + R(1/6)
sage: p2.tropical_variety().vertices()
{(1, -1/2), (7/6, -1/3)}
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = x + y
>>> p1.tropical_variety().vertices()
set()
>>> p2 = R(-Integer(2))*x**Integer(2) + R(-Integer(1))*x + R(Integer(1)/
↪Integer(2))*y + R(Integer(1)/Integer(6))
>>> p2.tropical_variety().vertices()
{(1, -1/2), (7/6, -1/3)}
```

weight_vectors()

Return the weight vectors for all vertices of `self`.

Weight vectors are a list of vectors associated with each vertex of the curve. Each vector corresponds to an edge emanating from that vertex and points in the direction of the edge.

Suppose v is a vertex adjacent to the edges $e_1, \ldots, e_k$ with respective weights $w_1, \ldots, w_k$. Every edge e_i is contained in a line (component) defined by an equation. Therefore, there exists a unique integer vector $v_i = (\alpha, \beta)$ in the direction of e_i such that $\gcd(\alpha, \beta) = 1$. Then, each vertex v yield the vectors $w_1 v_1, \ldots, w_k v_k$. These vectors will satisfy the following balancing condition: $\sum_{i=1}^{k} w_i v_i = 0$.

OUTPUT:

A dictionary where the keys represent the vertices, and the values are lists of vectors.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = R(-2)*x^2 + R(-1)*x + R(1/2)*y + R(1/6)
sage: p1.tropical_variety().weight_vectors()
{(1, -1/2): [(0, 1), (-1, -2), (1, 1)],
 (7/6, -1/3): [(-1, -1), (0, 1), (1, 0)]}

sage: p2 = R(2)*x^2 + x*y + R(2)*y^2 + x + R(-1)*y + R(3)
sage: p2.tropical_variety().weight_vectors()
{(-2, 0): [(-1, -1), (0, 1), (1, 0)],
 (-1, -3): [(-1, -1), (0, 1), (1, 0)],
 (-1, 0): [(-1, 0), (0, -1), (1, 1)],
 (3, 4): [(-1, -1), (0, 1), (1, 0)]}
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = R(-Integer(2))*x**Integer(2) + R(-Integer(1))*x + R(Integer(1)/
↪Integer(2))*y + R(Integer(1)/Integer(6))
>>> p1.tropical_variety().weight_vectors()
{(1, -1/2): [(0, 1), (-1, -2), (1, 1)],
 (7/6, -1/3): [(-1, -1), (0, 1), (1, 0)]}

>>> p2 = R(Integer(2))*x**Integer(2) + x*y + R(Integer(2))*y**Integer(2) + x␣
↪+ R(-Integer(1))*y + R(Integer(3))
>>> p2.tropical_variety().weight_vectors()
{(-2, 0): [(-1, -1), (0, 1), (1, 0)],
```

(continues on next page)

(continued from previous page)

```
 (-1, -3): [(-1, -1), (0, 1), (1, 0)],
 (-1, 0): [(-1, 0), (0, -1), (1, 1)],
 (3, 4): [(-1, -1), (0, 1), (1, 0)]}
```

class sage.rings.semirings.tropical_variety.**TropicalSurface**(*poly*)

Bases: *TropicalVariety*

A tropical surface in $\mathbf{R}^3$.

The tropical surface consists of planar regions and facets, which we can call cells. These cells are connected in such a way that they form a piecewise linear structure embedded in three-dimensional space. These cells meet along edges, where the balancing condition is satisfied. This balancing condition ensures that the sum of the outgoing normal vectors at each edge is zero, reflecting the equilibrium.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x,y,z> = PolynomialRing(T)
sage: p1 = x + y + z + R(0)
sage: tv = p1.tropical_variety(); tv
Tropical surface of 0*x + 0*y + 0*z + 0
sage: tv.components()
[[(t1, t1, t2), [t2 <= t1, 0 <= t1], 1],
[(t1, t2, t1), [max(0, t2) <= t1], 1],
[(0, t1, t2), [t2 <= 0, t1 <= 0], 1],
[(t1, t2, t2), [max(0, t1) <= t2], 1],
[(t1, 0, t2), [t2 <= 0, t1 <= 0], 1],
[(t1, t2, 0), [t1 <= 0, t2 <= 0], 1]]
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x', 'y', 'z',)); (x, y, z,) = R._first_ngens(3)
>>> p1 = x + y + z + R(Integer(0))
>>> tv = p1.tropical_variety(); tv
Tropical surface of 0*x + 0*y + 0*z + 0
>>> tv.components()
[[(t1, t1, t2), [t2 <= t1, 0 <= t1], 1],
[(t1, t2, t1), [max(0, t2) <= t1], 1],
[(0, t1, t2), [t2 <= 0, t1 <= 0], 1],
[(t1, t2, t2), [max(0, t1) <= t2], 1],
[(t1, 0, t2), [t2 <= 0, t1 <= 0], 1],
[(t1, t2, 0), [t1 <= 0, t2 <= 0], 1]]
```

plot(*color='random'*)

Return the plot of `self` by constructing a polyhedron from vertices in `self.polygon_vertices()`.

INPUT:

- `color` – string or tuple that represent a color (default: `random`); `random` means each polygon will be assigned a different color. If instead a specific `color` is provided, then all polygon will be given the same color.

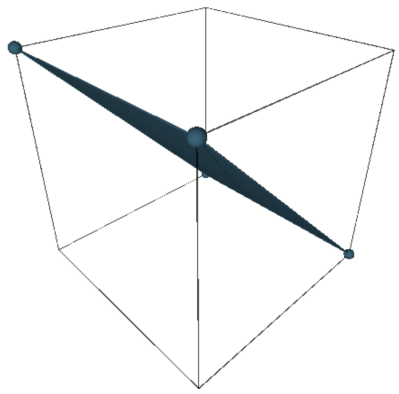
OUTPUT: Graphics3d Object

EXAMPLES:

A tropical surface that consist of only one cell:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y,z> = PolynomialRing(T)
sage: p1 = x + z
sage: tv = p1.tropical_variety()
sage: tv.plot()
Graphics3d Object
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y', 'z',)); (x, y, z,) = R._first_
↪ngens(3)
>>> p1 = x + z
>>> tv = p1.tropical_variety()
>>> tv.plot()
Graphics3d Object
```



A tropical surface with multiple cells that exhibit complex and intriguing geometric structures:

```
sage: p2 = x^2 + x + y + z + R(1)
sage: tv = p2.tropical_variety()
sage: tv.plot() # long time
Graphics3d Object
```

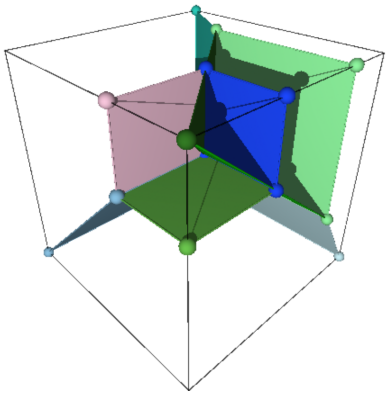
```
>>> from sage.all import *
>>> p2 = x**Integer(2) + x + y + z + R(Integer(1))
>>> tv = p2.tropical_variety()
>>> tv.plot() # long time
Graphics3d Object
```

class sage.rings.semirings.tropical_variety.**TropicalVariety**(*poly*)

Bases: UniqueRepresentation, SageObject

A tropical variety in $\mathbf{R}^n$.

A tropical variety is defined as a corner locus of tropical polynomial function. This means it consist of all points in $\mathbf{R}^n$ for which the minimum (maximum) of the function is attained at least twice.



We represent the tropical variety as a list of lists, where the inner list consist of three parts. The first one is a parametric equations for tropical roots. The second one is the condition for parameters. The third one is the order of the corresponding component.

INPUT:

- `poly` – a *TropicalMPolynomial*

ALGORITHM:

We need to determine a corner locus of this tropical polynomial function, which is all points $(x_1, x_2, \ldots, x_n)$ for which the maximum (minimum) is obtained at least twice. First, we convert each monomial to its corresponding linear function. Then for each two monomials of polynomial, we find the points where their values are equal. Since we attempt to solve the equality of two equations in n variables, the solution set will be described by $n - 1$ parameters.

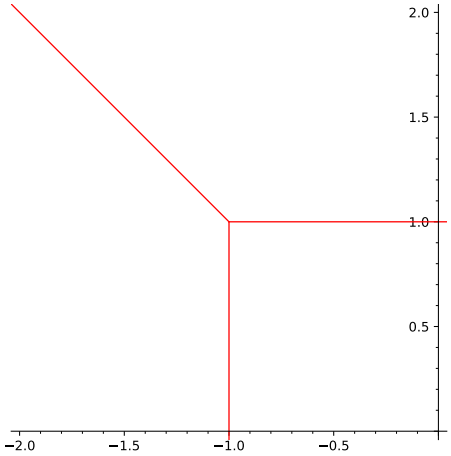
Next, we need to check if the value of previous two monomials at the points in solution set is really the maximum (minimum) of function. We do this by solving the inequality of the previous monomial with all other monomials in the polynomial after substituting the parameter. This will give us the condition of parameters. Each of this condition is then combined by union operator. If this final condition is not an empty set, then it represent one component of tropical root. Then we calculate the weight of this particular component by the maximum of gcd of the numbers $|i - k|$ and $|j - l|$ for all pairs (i, j) and (k, l) such that the value of on this component is given by the corresponding monomials.

EXAMPLES:

We construct a tropical variety in $\mathbf{R}^2$, where it is called a tropical curve:

```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = R(1)*x + x*y + R(0); p1
0*x*y + 1*x + 0
sage: tv = p1.tropical_variety(); tv
Tropical curve of 0*x*y + 1*x + 0
sage: tv.components()
[[(t1, 1), [t1 >= -1], 1], [(-1, t1), [t1 <= 1], 1], [(-t1, t1), [t1 >= 1], 1]]
sage: tv.vertices()
{(-1, 1)}
sage: tv.plot()
Graphics object consisting of 3 graphics primitives
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = R(Integer(1))*x + x*y + R(Integer(0)); p1
0*x*y + 1*x + 0
>>> tv = p1.tropical_variety(); tv
Tropical curve of 0*x*y + 1*x + 0
>>> tv.components()
[[(t1, 1), [t1 >= -1], 1], [(-1, t1), [t1 <= 1], 1], [(-t1, t1), [t1 >= 1], 1]]
>>> tv.vertices()
{(-1, 1)}
>>> tv.plot()
Graphics object consisting of 3 graphics primitives
```

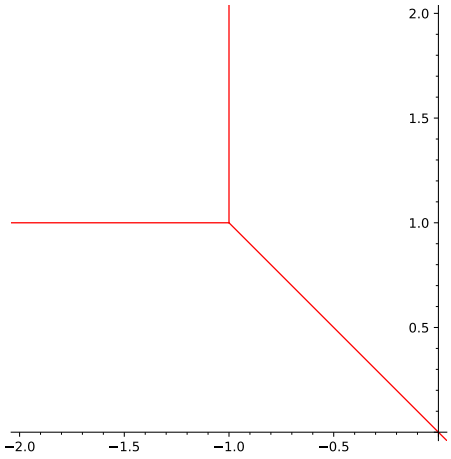


A slightly different result will be obtained if we use min-plus algebra for the base tropical semiring:

```
sage: T = TropicalSemiring(QQ, use_min=True)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = R(1)*x + x*y + R(0)
sage: tv = p1.tropical_variety(); tv
Tropical curve of 0*x*y + 1*x + 0
sage: tv.components()
[[(t1, 1), [t1 <= -1], 1], [(-1, t1), [t1 >= 1], 1], [(-t1, t1), [t1 <= 1], 1]]
sage: tv.plot()
Graphics object consisting of 3 graphics primitives
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=True)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = R(Integer(1))*x + x*y + R(Integer(0))
>>> tv = p1.tropical_variety(); tv
Tropical curve of 0*x*y + 1*x + 0
>>> tv.components()
[[(t1, 1), [t1 <= -1], 1], [(-1, t1), [t1 >= 1], 1], [(-t1, t1), [t1 <= 1], 1]]
>>> tv.plot()
Graphics object consisting of 3 graphics primitives
```

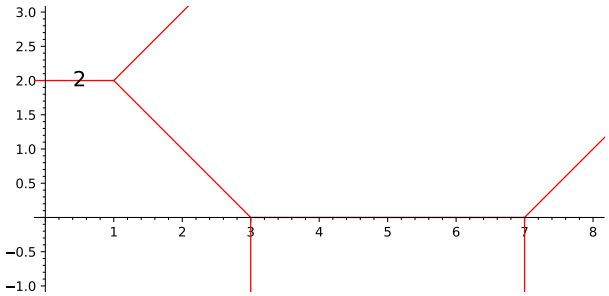
Tropical variety can consist of multiple components with varying orders:




```
sage: T = TropicalSemiring(QQ, use_min=False)
sage: R.<x,y> = PolynomialRing(T)
sage: p1 = R(7) + T(4)*x + y + R(4)*x*y + R(3)*y^2 + R(-3)*x^2
sage: tv = p1.tropical_variety(); tv
Tropical curve of (-3)*x^2 + 4*x*y + 3*y^2 + 4*x + 0*y + 7
sage: tv.components()
[[(3, t1), [t1 <= 0], 1],
[(-t1 + 3, t1), [0 <= t1, t1 <= 2], 1],
[(t1, 2), [t1 <= 1], 2],
[(t1, 0), [3 <= t1, t1 <= 7], 1],
[(7, t1), [t1 <= 0], 1],
[(t1 - 1, t1), [2 <= t1], 1],
[(t1 + 7, t1), [0 <= t1], 1]]
sage: tv.plot()
Graphics object consisting of 8 graphics primitives
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ, use_min=False)
>>> R = PolynomialRing(T, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> p1 = R(Integer(7)) + T(Integer(4))*x + y + R(Integer(4))*x*y +␣
↪R(Integer(3))*y**Integer(2) + R(-Integer(3))*x**Integer(2)
>>> tv = p1.tropical_variety(); tv
Tropical curve of (-3)*x^2 + 4*x*y + 3*y^2 + 4*x + 0*y + 7
>>> tv.components()
[[(3, t1), [t1 <= 0], 1],
[(-t1 + 3, t1), [0 <= t1, t1 <= 2], 1],
[(t1, 2), [t1 <= 1], 2],
[(t1, 0), [3 <= t1, t1 <= 7], 1],
[(7, t1), [t1 <= 0], 1],
[(t1 - 1, t1), [2 <= t1], 1],
[(t1 + 7, t1), [0 <= t1], 1]]
>>> tv.plot()
Graphics object consisting of 8 graphics primitives
```

If the tropical polynomial have $n > 2$ variables, then the result will be a tropical hypersurface embedded in a real space $\mathbf{R}^n$:




```
sage: T = TropicalSemiring(QQ)
sage: R.<w,x,y,z> = PolynomialRing(T)
sage: p1 = x*y + R(-1/2)*x*z + R(4)*z^2 + w*x
sage: tv = p1.tropical_variety(); tv
Tropical hypersurface of 0*w*x + 0*x*y + (-1/2)*x*z + 4*z^2
sage: tv.components()
[[(t1, t2, t3 - 1/2, t3), [t2 - 9/2 <= t3, t3 <= t1 + 1/2, t2 - 5 <= t1], 1],
[(t1, 2*t2 - t3 + 4, t3, t2), [t3 + 1/2 <= t2, t3 <= t1], 1],
[(t1, t2, t1, t3), [max(t1 + 1/2, 1/2*t1 + 1/2*t2 - 2) <= t3], 1],
[(t1, t2 + 9/2, t3, t2), [t2 <= min(t3 + 1/2, t1 + 1/2)], 1],
[(t1 - 1/2, t2, t3, t1), [t2 - 9/2 <= t1, t1 <= t3 + 1/2, t2 - 5 <= t3], 1],
[(2*t1 - t2 + 4, t2, t3, t1), [t1 <= min(1/2*t2 + 1/2*t3 - 2, t2 - 9/2)], 1]]
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('w', 'x', 'y', 'z',)); (w, x, y, z,) = R._first_
↪ngens(4)
>>> p1 = x*y + R(-Integer(1)/Integer(2))*x*z + R(Integer(4))*z**Integer(2) + w*x
>>> tv = p1.tropical_variety(); tv
Tropical hypersurface of 0*w*x + 0*x*y + (-1/2)*x*z + 4*z^2
>>> tv.components()
[[(t1, t2, t3 - 1/2, t3), [t2 - 9/2 <= t3, t3 <= t1 + 1/2, t2 - 5 <= t1], 1],
[(t1, 2*t2 - t3 + 4, t3, t2), [t3 + 1/2 <= t2, t3 <= t1], 1],
[(t1, t2, t1, t3), [max(t1 + 1/2, 1/2*t1 + 1/2*t2 - 2) <= t3], 1],
[(t1, t2 + 9/2, t3, t2), [t2 <= min(t3 + 1/2, t1 + 1/2)], 1],
[(t1 - 1/2, t2, t3, t1), [t2 - 9/2 <= t1, t1 <= t3 + 1/2, t2 - 5 <= t3], 1],
[(2*t1 - t2 + 4, t2, t3, t1), [t1 <= min(1/2*t2 + 1/2*t3 - 2, t2 - 9/2)], 1]]
```

components()

Return all components of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<a,x,y,z> = PolynomialRing(T)
sage: tv = (a+x+y+z).tropical_variety()
sage: tv.components()
```

(continues on next page)

(continued from previous page)

```
[[(t1, t1, t2, t3), [t1 <= min(t3, t2)], 1],
 [(t1, t2, t1, t3), [t1 <= t3, t1 <= t2], 1],
 [(t1, t2, t3, t1), [t1 <= min(t3, t2)], 1],
 [(t1, t2, t2, t3), [t2 <= t3, t2 <= t1], 1],
 [(t1, t2, t3, t2), [t2 <= min(t3, t1)], 1],
 [(t1, t2, t3, t3), [t3 <= min(t1, t2)], 1]]
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('a', 'x', 'y', 'z',)); (a, x, y, z,) = R._
↪first_ngens(4)
>>> tv = (a+x+y+z).tropical_variety()
>>> tv.components()
[[(t1, t1, t2, t3), [t1 <= min(t3, t2)], 1],
 [(t1, t2, t1, t3), [t1 <= t3, t1 <= t2], 1],
 [(t1, t2, t3, t1), [t1 <= min(t3, t2)], 1],
 [(t1, t2, t2, t3), [t2 <= t3, t2 <= t1], 1],
 [(t1, t2, t3, t2), [t2 <= min(t3, t1)], 1],
 [(t1, t2, t3, t3), [t3 <= min(t1, t2)], 1]]
```

dimension()

Return the dimension of `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<a,x,y,z> = PolynomialRing(T)
sage: p1 = x*y + R(-1)*x*z
sage: p1.tropical_variety().dimension()
4
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('a', 'x', 'y', 'z',)); (a, x, y, z,) = R._
↪first_ngens(4)
>>> p1 = x*y + R(-Integer(1))*x*z
>>> p1.tropical_variety().dimension()
4
```

number_of_components()

Return the number of components that make up `self`.

EXAMPLES:

```
sage: T = TropicalSemiring(QQ)
sage: R.<a,x,y,z> = PolynomialRing(T)
sage: p1 = x*y*a + x*z + y^2 + a*x + y + z
sage: p1.tropical_variety().number_of_components()
13
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
```

(continues on next page)

(continued from previous page)

```
>>> R = PolynomialRing(T, names=('a', 'x', 'y', 'z',)); (a, x, y, z,) = R._
↪first_ngens(4)
>>> p1 = x*y*a + x*z + y**Integer(2) + a*x + y + z
>>> p1.tropical_variety().number_of_components()
13
```

weight_vectors()

Return the weight vectors for each unique intersection of components of `self`.

Weight vectors are a list of vectors associated with each unique intersection of the components of tropical variety. Each vector is a normal vector to a component with respect to the unique intersection lying within that component.

Assume `self` is a n-dimensional tropical variety. Suppose L is an intersection lying within the components $S_1, \ldots, S_k$ with respective weights $w_1, \ldots, w_k$. This L is a linear structure in $\mathbf{R}^{n-1}$ and has $n-1$ direction vectors $d_1, d_2, \ldots, d_{n-1}$. Each component $S_1, \ldots, S_k$ has a normal vector $n_1, \ldots, n_k$. Then, we scale each normal vector to an integer vector such that the greatest common divisor of its elements is 1.

The weight vector of a component S_i with respect to L can be found by calculating the cross product between direction vectors of L and normal vector n_i.These vectors will satisfy the balancing condition $\sum_{i=1}^{k} w_k v_k = 0$.

OUTPUT:

A tuple of three dictionaries. The first dictionary contains equations representing the intersections. The second dictionary contains indices of components that contains the intersection. The third dictionary contains lists of vectors.

EXAMPLES:

Weight vectors of tropical surface:

```
sage: T = TropicalSemiring(QQ)
sage: R.<x,y,z> = PolynomialRing(T)
sage: p = x^2 + R(-1)*y + z + R(1)
sage: tv = p.tropical_variety()
sage: tv.weight_vectors()
({0: ((1/2*u2, u2 + 1, u2), {u2 <= 1}),
  1: ((1/2, 2, u2), {1 <= u2}),
  2: ((1/2, u2, 1), {2 <= u2}),
  3: ((u1, 2, 1), {(1/2) <= u1})},
 {0: [0, 1, 3], 1: [0, 2, 4], 2: [1, 2, 5], 3: [3, 4, 5]},
 {0: [(1, 2, -5/2), (1, -5/2, 2), (-2, 1/2, 1/2)],
  1: [(-1, -2, 0), (0, 2, 0), (1, 0, 0)],
  2: [(1, 0, 2), (0, 0, -2), (-1, 0, 0)],
  3: [(0, 1, 1), (0, 0, -1), (0, -1, 0)]})
```

```
>>> from sage.all import *
>>> T = TropicalSemiring(QQ)
>>> R = PolynomialRing(T, names=('x', 'y', 'z',)); (x, y, z,) = R._first_
↪ngens(3)
>>> p = x**Integer(2) + R(-Integer(1))*y + z + R(Integer(1))
>>> tv = p.tropical_variety()
>>> tv.weight_vectors()
({0: ((1/2*u2, u2 + 1, u2), {u2 <= 1}),
```

(continues on next page)

(continued from previous page)

```
 1: ((1/2, 2, u2), {1 <= u2}),
 2: ((1/2, u2, 1), {2 <= u2}),
 3: ((u1, 2, 1), {(1/2) <= u1})},
{0: [0, 1, 3], 1: [0, 2, 4], 2: [1, 2, 5], 3: [3, 4, 5]},
{0: [(1, 2, -5/2), (1, -5/2, 2), (-2, 1/2, 1/2)],
 1: [(-1, -2, 0), (0, 2, 0), (1, 0, 0)],
 2: [(1, 0, 2), (0, 0, -2), (-1, 0, 0)],
 3: [(0, 1, 1), (0, 0, -1), (0, -1, 0)]})
```

CHAPTER

SEVEN

INDICES AND TABLES

- Index
- Module Index
- Search Page

PYTHON MODULE INDEX

r

sage.rings.semirings.non_negative_integer_semiring, 1
sage.rings.semirings.tropical_matrix, 45
sage.rings.semirings.tropical_mpolynomial, 25
sage.rings.semirings.tropical_polynomial, 9
sage.rings.semirings.tropical_semiring, 3
sage.rings.semirings.tropical_variety, 49

INDEX

A

additive_identity() (*sage.rings.semirings.tropical_semiring.TropicalSemiring method*), 5
additive_semigroup_generators() (*sage.rings.semirings.non_negative_integer_semiring.NonNegativeIntegerSemiring method*), 2

C

components() (*sage.rings.semirings.tropical_variety.TropicalVariety method*), 63
contribution() (*sage.rings.semirings.tropical_variety.TropicalCurve method*), 51

D

dimension() (*sage.rings.semirings.tropical_variety.TropicalVariety method*), 64
dual_subdivision() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomial method*), 28

E

Element (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomialSemiring attribute*), 41
Element (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring attribute*), 18
Element (*sage.rings.semirings.tropical_semiring.TropicalSemiring attribute*), 5
extremum_cycle_mean() (*sage.rings.semirings.tropical_matrix.Matrix_tropical_dense method*), 45

F

factor() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomial method*), 12

G

gen() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomialSemiring method*), 41
gen() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring method*), 19
gens() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomialSemiring method*), 41
gens() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring method*), 19
gens() (*sage.rings.semirings.tropical_semiring.TropicalSemiring method*), 5
genus() (*sage.rings.semirings.tropical_variety.TropicalCurve method*), 52

I

infinity() (*sage.rings.semirings.tropical_semiring.TropicalSemiring method*), 5
interpolation() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring method*), 19
is_simple() (*sage.rings.semirings.tropical_variety.TropicalCurve method*), 52
is_smooth() (*sage.rings.semirings.tropical_variety.TropicalCurve method*), 53
is_sparse() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring method*), 20

K

kleene_star() (*sage.rings.semirings.tropical_matrix.Matrix_tropical_dense method*), 46

L

lift() (*sage.rings.semirings.tropical_semiring.TropicalSemiringElement method*), 6

M

Matrix_tropical_dense (*class in sage.rings.semirings.tropical_matrix*), 45
module
    sage.rings.semirings.non_negative_integer_semiring, 1
    sage.rings.semirings.tropical_matrix, 45
    sage.rings.semirings.tropical_mpolynomial, 25
    sage.rings.semirings.tropical_polynomial, 9

sage.rings.semirings.tropical_semiring, 3
sage.rings.semirings.tropical_variety, 49
multiplicative_identity() (*sage.rings.semirings.tropical_semiring.TropicalSemiring method*), 5
multiplicative_order() (*sage.rings.semirings.tropical_semiring.TropicalSemiringElement method*), 6

N

newton_polytope() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomial method*), 34
ngens() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomialSemiring method*), 41
ngens() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring method*), 21
NonNegativeIntegerSemiring (*class in sage.rings.semirings.non_negative_integer_semiring*), 1
number_of_components() (*sage.rings.semirings.tropical_variety.TropicalVariety method*), 64

O

one() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomialSemiring method*), 42
one() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring method*), 21
one() (*sage.rings.semirings.tropical_semiring.TropicalSemiring method*), 5

P

piecewise_function() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomial method*), 13
plot() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomial method*), 14
plot() (*sage.rings.semirings.tropical_variety.TropicalCurve method*), 53
plot() (*sage.rings.semirings.tropical_variety.TropicalSurface method*), 58
plot3d() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomial method*), 36

R

random_element() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomialSemiring method*), 42
random_element() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring method*), 21
roots() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomial method*), 16

S

sage.rings.semirings.non_negative_integer_semiring
module, 1
sage.rings.semirings.tropical_matrix
module, 45
sage.rings.semirings.tropical_mpolynomial
module, 25
sage.rings.semirings.tropical_polynomial
module, 9
sage.rings.semirings.tropical_semiring
module, 3
sage.rings.semirings.tropical_variety
module, 49
split_form() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomial method*), 17
strong_transitive_closure() (*sage.rings.semirings.tropical_matrix.Matrix_tropical_dense method*), 46
subs() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomial method*), 37

T

term_order() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomialSemiring method*), 43
tropical_variety() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomial method*), 39
TropicalCurve (*class in sage.rings.semirings.tropical_variety*), 49
TropicalMPolynomial (*class in sage.rings.semirings.tropical_mpolynomial*), 25
TropicalMPolynomialSemiring (*class in sage.rings.semirings.tropical_mpolynomial*), 40
TropicalPolynomial (*class in sage.rings.semirings.tropical_polynomial*), 9
TropicalPolynomialSemiring (*class in sage.rings.semirings.tropical_polynomial*), 18
TropicalSemiring (*class in sage.rings.semirings.tropical_semiring*), 3
TropicalSemiringElement (*class in sage.rings.semirings.tropical_semiring*), 6
TropicalSurface (*class in sage.rings.semirings.tropical_variety*), 58
TropicalToTropical (*class in sage.rings.semirings.tropical_semiring*), 7
TropicalVariety (*class in sage.rings.semirings.tropical_variety*), 59

V

vertices() (*sage.rings.semirings.tropical_variety.TropicalCurve method*), 56

W

weak_transitive_closure() (*sage.rings.semirings.tropical_matrix.Matrix_tropical_dense method*), 47
weight_vectors() (*sage.rings.semirings.tropical_variety.TropicalCurve method*), 57
weight_vectors() (*sage.rings.semirings.tropical_variety.TropicalVariety method*), 65

Z

zero() (*sage.rings.semirings.tropical_mpolynomial.TropicalMPolynomialSemiring method*), 43
zero() (*sage.rings.semirings.tropical_polynomial.TropicalPolynomialSemiring method*), 23
zero() (*sage.rings.semirings.tropical_semiring.TropicalSemiring method*), 6